

Technical Survey Report on NBAccess, ClaudeRuntime, ClaudeOrchestrator, and SourceVault

Survey basis date: July 16, 2026 (revised: July 17, 2026)

Contents

Executive Summary

How to Read This Report — Terminology, Maturity Labels, and Authoritative Sources

 Maturity Labels and Authoritative Sources

 Mini-glossary

 Cross-layer data type correspondence

Chapter 1. Survey Targets and System Overview

 1.1 Survey Targets

 1.2 Four-Layer Architecture

 1.3 Primary Anticipated Use Cases

 1.4 claudecode — The Integration and Compatibility Layer Behind the Scenes

 1.5 Usage Has Expanded from "One Question, One Answer" to "Knowledge, Workflows, and Services"

 1.6 Future Directions

Chapter 2. NBAccess — The Final Boundary for Notebooks and Confidential Data

 2.1 Role and Main Responsibilities

 2.2 Key APIs

 2.3 Privacy and Dependency Propagation

 2.4 File Privacy and the CloudPublishable Boundary

 2.5 Safe Expression Execution

 2.6 Cryptography Layer

 2.7 Assessment

 2.8 Future Directions

Chapter 3. ClaudeRuntime — The Expression-Proposal Loop

 3.1 Role

 3.2 Key Files

 3.3 Adapter Model

 3.4 Turn Execution and the Decision Model

 3.5 Diversity of Entry Points — The Boundary Between ClaudeEval and the PromptRouter

 3.6 Task Placement

 3.7 External Process Management

 3.8 Assessment

 3.9 Future Directions

Chapter 4. ClaudeOrchestrator — Multi-Agent Orchestration and the Single Committer

 4.1 Role

 4.2 The Core Design Shift — Separating Workers from the Committer

 4.3 Key Files

 4.4 Roles and Capabilities

 4.5 Execution Pipeline — Planning, Spawn, Reduce, Commit

 4.6 Petri Net Workflow

 4.7 PromptWorkflow — Converting Complex Natural-Language Requests into Workflow Candidates

 4.8 Turning the Process from Generating Specifications from Requirements to Implementation into Workflows

 4.9 Assessment

 4.10 Future Directions

Chapter 5. Foundational Architecture of SourceVault

 5.1 Source-First Design — Source, Snapshot, and Derived Artifact

5.2 Module Structure

5.3 Ingest Flow

5.4 Claims and Evidence Bundles

5.5 PromptRouter and Deterministic Usage Paths

5.6 Release Context and Fail-Closed Search

5.7 sv:// URI — Canonical Identity Used Across SourceVault

5.8 SourceVault MCP — A Gateway for External LLMs as a Projection Layer

5.9 MCP Sharing of Directives and Artifact Deposit

5.10 Assessment

5.11 Future Directions

Chapter 6. Code-Level Integration Analysis

6.1 Design That Concentrates Safety at the Lower Boundaries

6.2 Separation of Proposal and Execution

6.3 Separation of Artifacts and Live State

6.4 Immutable and Append-Only

6.5 Task Placement and Resource Places

6.6 Responsibility Boundaries Across the Four Repositories

6.7 Future Directions

Chapter 7. Evolution from the Four-Layer Stack into a Knowledge-and-Execution Operating Model

7.1 Prompt, Workflow, MCP, and SourceVault Form a Closed Loop

7.2 SourceVault_autotrigger — Launching Workflows Automatically and Safely

7.3 Designing SystemDoctor as the First Automated Workflow

7.4 Sharing SourceVault Across Multiple PCs

7.5 What Redundancy and Load Balancing Have Achieved So Far

7.6 Future Directions

Chapter 8. The SourceVault Mining Layer

8.1 Definition of Mining

8.2 The Multi-Stage Structure of Search

8.3 Neighborhood Search and Neighbor Hopping

8.4 Cross-Medium Common Projection and Identity

8.5 The OOPS Corpus — Initial Reference Coordinates of the Knowledge Space

8.6 Extension to General Mail and Topic Growth

8.7 Privacy Boundaries

8.8 Implementation Status

8.9 Future Directions

Chapter 9. Defense in Depth Across Safety, Diagnostics, and Cognitive Support

9.1 Why a safety layer is needed outside the mining layer

9.2 Ingress boundary — quarantine of untrusted data and taint tracking

9.3 Egress boundary — sensitive cells and prompt privacy

9.4 LLM submission and execution boundary — request binding and capabilities

9.5 Diagnostics — SystemDoctor and the confidentiality of diagnostic information

9.6 Cognitive support

9.7 Operational state anomaly — observing non-execution and forgetting

9.8 Routine Agenda — treating Calendar, Notebook, and Mail on the same attention surface

9.9 Adjudication and anomaly analysis

9.10 Summary of defense in depth and boundaries after autonomization

9.11 Future Directions

Chapter 10. Comparison with Similar Systems

10.1 Comparison Axes

10.2 Comparison Table

10.3 Comparison with LangGraph

10.4 Comparison with Microsoft Agent Framework

10.5 Comparison with Haystack

10.6 Positioning of the transreal stack

10.7 Future Directions

Chapter 11. Maturity, Issues, and Future Priorities

11.1 Maturity Should Not Be Assessed as a Single Score

11.2 Module Scale of SourceVault

11.3 Security Priorities

11.4 Issues as Public OSS

11.5 Recommended Staged Consolidation [Author's proposal]

11.6 Future Directions

Chapter 12. Overall Assessment

12.1 Baseline Assessment

12.2 Assessment Including the Mining and Safety Layers

12.3 Technical Value

12.4 Limitations

12.5 Final Conclusions

12.6 Future Directions

Appendix A. Main Reference Materials

Appendix B. Main Code, API, and Design Documents Referenced in the Expansion and Revision

How to Read the Implementation Status

Appendix C. Discrepancies Between Source Code and API Documentation (verified July 17, 2026)

Executive Summary

The four public GitHub repositories covered by this survey—NBAccess, ClaudeRuntime, ClaudeOrchestrator, and SourceVault—constitute a layered system built on Wolfram Language/Mathematica as its execution platform. NBAccess serves as the final boundary for Notebooks, confidentiality, and safe execution; ClaudeRuntime handles single LLM turns and the Expression-Proposal loop; ClaudeOrchestrator is responsible for multiple workers, workflows, and the single committer; and SourceVault covers source lifecycle, evidence, search, and long-term knowledge.

Layer	Core responsibilities	Representative boundary
NBAccess	Notebook semantic API, privacy, held expression validation, cryptographic keys	Final boundary for execution and secrets
ClaudeRuntime	Single agent turn, provider adapters, external runners, task placement	Proposals are never evaluated directly as commands
ClaudeOrchestrator	Planning, workers, Petri-net workflows, reduce, commit	Workers are artifact producers; the Notebook is the single committer
SourceVault	source/snapshot/claim, URIs, search, MCP, Knowledge Home	Knowledge sharing with release gates and provenance maintained

In practice, however, looking at these four layers alone is not sufficient. Although it rarely comes to the forefront, transreal/claudecode still retains a large number of fundamental functions—ClaudeEval, multi-provider integration, Claude Directives, the shared polling tick, the palette, MCP server registration, and more—and remains important as the compatibility and glue layer that underpins the current integrated environment. Even as responsibilities continue to migrate toward ClaudeRuntime/ClaudeOrchestrator, understanding current operation requires taking claudecode into account as background infrastructure.

Usage paths are likewise not limited to "propose a Wolfram expression from a prompt via ClaudeEval, then validate and execute it through NBAccess." The SourceVault PromptRouter can resolve saved or deterministic PromptRoutes before any LLM call and propose unevaluated expressions to ClaudeEval. Complex prompts are treated by ClaudeOrchestrator PromptWorkflow as WorkflowNet candidates, and new workflows stop at NeedsApproval even after safe parsing and validation. In addition, SourceVault implements spec-review, which builds consensus on a design specification from requirements in a Notebook, and spec-impl, which generates and verifies a coded workflow from an approved specification—turning development itself into a workflow: requirements → specification → implementation → verification → catalog registration.

SourceVault is converging on referencing ingested sources not as mere file paths but—for external APIs, search results, and relationship descriptions—via canonical URIs of the form sv://.... Categories such as mail/web/pdf are expressed not as URI namespaces but as Class/MediaType/Kind sidecar metadata, using identity namespaces such as object, snapshot, artifact, chunk, file, and packageapi. Legacy internal references such as sourceId, snapshot:..., and blob:... remain for compatibility, but for exchanges that cross MCP or the Orchestrator, sv:// URIs serve as the stable opaque references.

SourceVault MCP is what safely opens this URI layer to external LLMs. The current MCP has 26 tools, providing catalog/search/get, access requests, runtime capabilities, web search, mail/OOPS search, GitHub commit logs, lookup of the rules/skills of Claude Directives, append-only artifact deposit, and more. While Claude Code and Codex can use the Claude Directives deployed into their local harnesses, publishing the same rules/skills as an MCP database allows MCP-capable local models such as those in LM Studio to reference the same set of rules and skills. Furthermore, because LLMs can create sv://artifact/... via sourcevault_deposit, SourceVault can be used as an artifact exchange surface between LLMs and workers. This is compatible with the single-committer principle, under which workers cannot write directly to live Notebooks.

On the autonomous-execution side, SourceVault_autotrigger monitors schedules, SourceVault events, and conditions to construct jobs, and can launch workflows and CatalogWorkflows asynchronously. The design specification designates the first automatically executed workflow to be a comprehensive health check by SourceVaultSystemDoctor/diagnostics, with general workflows presupposing that the health of the required components is OK or an accepted Degraded state. In the current implementation, the dispatch gate based on a lightweight doctor, machine heartbeat, specific-machine dispatch,

and remote result publication are implemented, whereas automatic triggering of the comprehensive doctor and fully automatic load balancing/failover of the WorkerPool remain incomplete.

In configurations where multiple PCs share the same SourceVault, remote workflow execution on a specific PC and sharing of the results are already possible by combining per-machine single-writer jobs/runs logs, machine heartbeats, aggregator rollups, SpecificMachine placement, atomic dispatch claims, headless dispatch, and license seat guards. These form the foundation for redundancy and load balancing, but because health-aware automatic WorkerPool selection and full failover still include design-stage elements, implemented functionality and the future vision must be evaluated separately.

Taken together, the four main repositories plus the auxiliary claudecode layer are not merely a set of tools for calling LLMs from a Notebook. They have by now evolved into a layered operating model for Wolfram Notebooks that connects knowledge acquisition, URI-based referencing, structuring, search, mining, LLM inference, workflow generation, autonomous triggering, multi-PC execution, and auditing while maintaining privacy and capability boundaries.

Primary code and documentation references: SourceVault_info/docs/user_manual.md / SourceVault_info/docs/api_promptrouter.md / SourceVault_info/docs/api_mcp.md / SourceVault_info/design/sourcevault_universal_mcp_access_spec_v2.md / SourceVault_info/design/sourcevault_auto_trigger_scheduler_spec_v0_1.md / claudecode_info/docs/api.md

How to Read This Report — Terminology, Maturity Labels, and Authoritative Sources

Because this report makes dense use of terminology specific to the systems under survey, before entering the main text we first summarize (1) the implementation maturity labels, (2) the authoritative sources for the descriptions, (3) a minimal glossary of terms, and (4) the correspondence between data types that appear across layers.

Maturity Labels and Authoritative Sources

Descriptions of implementation status use the following four-level labels.

Label	Meaning
[Implemented]	Implementation can be confirmed in the current source code (a static check; it does not mean behavioral testing in which the tests were re-run for this survey)
[Partially implemented]	Only some paths are implemented; full wiring and enforcement are not yet complete
[Design-only]	Present in the design specifications but not yet implemented, or not yet wired, in the current code
[Author's proposal]	Not settled even in the specifications; an organization or proposal by the author of this report

The authoritative sources for the descriptions are, first, the current source code (each *.wl) and, second, the API documents in *_info/docs/api*.md; the specifications under *_info/design/ were used to confirm future direction and design intent. Where the source and the API documents contradict each other, this is called out explicitly in the main text as a discrepancy between primary sources, and a full list is provided in Appendix C. Figures such as tool counts and entrypoint counts are values confirmed by mechanical tallies from the source as of July 16–17, 2026. By contrast, the test green counts in the main text (test pass counts, e.g., "262 green") are quotations of values stated in the API documents, not values obtained by re-running the tests in this survey.

Mini-glossary

Term	Minimal explanation
artifact / live state / single committer	The distinction between intermediate products generated by workers and the authoritative state such as Notebooks. The principle that applying changes to live state is restricted to a single committer
Notebook semantic API / Held Expression	An API that treats a Notebook as Cell expressions and metadata rather than strings. A Held Expression is a Wolfram expression whose evaluation is suppressed with HoldComplete or the like
turn / proposal / adapter	A single unit of LLM dialogue; the unevaluated proposal an LLM returns; a set of functions (an Association) that connects a provider or execution method to the Runtime contract
Petri net (place / token / transition)	A formal model that represents a workflow as states (places), values flowing through the states (tokens), and transitions. Well suited to expressing approval waits and joins
source / snapshot / claim / chunk / span / Evidence Bundle	An original document; its immutable version at a point in time; an extracted claim; a search unit; a positional reference within a body of text; the set of grounds an artifact depends on
provenance / staleness	Tracking which sources, spans, and processing a product derives from. Staleness is the state in which the grounds have become outdated due to an upstream version update
release gate / ReleaseContext	The policy decision on whether search/retrieval results may be returned, and the registered profile used for that decision
sv:// URI / opaque reference	A stable reference identity that does not embed the content itself. The URI itself carries no authority. Distinguished from legacy refs used for internal implementation

Term	Minimal explanation
MCP / tool / projection layer / service kernel	Model Context Protocol. A tool is an operation exposed by MCP. SourceVault MCP is a layer that returns only permitted projections of internal objects, executed by a service kernel
claudecode / Claude Directives / shared polling tick	A compatibility and glue package outside the four layers. Directives is the canonical collection of CLAUDE.md, rules, and skills. The polling tick is a shared monitoring foundation that does not freeze the FE
PrivacyLevel / AccessLevel / PrivacySpec	The confidentiality of the data (0.0–1.0); the ceiling a recipient may receive; the Association holding those settings. The check is <code>PrivacyLevel <= AccessLevel</code>
task placement (main kernel / subkernel / wolframscript)	The classification of execution locations, which differ in Notebook UI dependence, secrets, license seats, and isolation
capability / CapabilityLease / PreparedInputToken / ExecutionTicket / AccessGrant	A label for a kind of side effect; a time-limited execution permit; a token binding the entire content sent to the LLM; a one-time execution proof issued at the moment a lease is consumed; an approval for data access
immutable / append-only / event / current projection	In areas implemented as event-sourced, records are appended rather than overwritten, and the current state is projected from the event sequence. Registries, sidecars, caches, and the like have other persistence contracts (see 6.4)
taint / quarantine / declassification / sanitized projection	Tracking the propagation of dangerous input into its derivatives; isolation; constrained release; a sanitized projection for publication
retrieval / grounding / primer / BM25 / dense / rerank	Candidate retrieval; grounding to the original text; a summary layer for narrowing candidates; lexical search; vector search; reranking
KG / Neighbor Hopping / TopicPosition / UnknownMass	Knowledge graph; moving from document to neighboring document; a document's position in topic space; the residual that cannot be explained by existing topics
Knowledge Home / OOPS	The core working surface for personal knowledge, and the private mailing-list corpus from 1992–2005 (a proper name) that serves as its initial coordinates
SystemDoctor (lightweight / comprehensive) / SIEM	The lightweight pre-dispatch health gate and the comprehensive diagnostics. SIEM is the aggregation and analysis layer for security events
WorkerPool / SpecificMachine / EnvironmentIndependent	Modes for selecting the execution PC. SpecificMachine is implemented; automatic load balancing for WorkerPool is not
Shadow / Warn / Enforce	Staged modes: observe only; warn and continue; reject. An observation toggle exists separately from the mode, and the two are configured independently
bitemporal / crypto-shredding / Kleene three-valued logic	Dual management of valid time and record time; effective erasure by key destruction; three-valued logic with True/False/Unknown
owner / authority / principal	The system's administrator in person; the authority whose instructions may be treated as commands; the authenticated calling subject
sidecar metadata	Auxiliary metadata held in separate files or under separate keys from the main record (URI or snapshot). Holds media type, resolution status, and the like without embedding them into the URI itself
harness (local / tool harness)	The set of local execution facilities that a CLI or worker uses for the filesystem, shell, Directives, and MCP tools
headless	Kernel/CLI execution without the FrontEnd/Notebook UI
CatalogWorkflow	An AutoTrigger execution kind that resolves a catalog-registered workflow by slug or the like and launches it
DAG	Directed acyclic graph. In this report it mainly represents TaskSpec dependencies and parallel executability
green	Means a test pass. "262 green" is the test pass count stated in the API documents
schema-only	A context projection that excludes value contents and passes only types, shapes, keys, and element counts

Term	Minimal explanation
fail-closed	The policy of defaulting to empty results or rejection—rather than permission—when context is insufficient or a decision fails
TOCTOU	Time-of-check/time-of-use: the problem of the target being swapped between check time and use time
idempotency key	An identifier that lets a resend of the same request be treated as a replay rather than executed a second time

Note that `$ClaudeAdvisaryModel`, which appears in the main text, is a real symbol name spelled exactly as shown (not a misspelling of Advisory) and refers to the verification model used in specification reviews.

Cross-layer data type correspondence

Family	Types	Owning layer
privacy / access policy	PrivacySpec (including AccessLevel) / AccessSpec and AccessRequest / ReleaseContext / AccessGrant	NBAccess / NBAccess and MCP / SourceVault searchindex / SourceVault MCP
execution permission tokens	CapabilityLease / PreparedInputToken / ExecutionTicket	SourceVault capbroker
tasks and artifacts	TaskSpec / Artifact / ReducedArtifact	ClaudeOrchestrator
units of knowledge	source / snapshot / object / chunk / claim / span / artifact	SourceVault

These are independent per-layer implementations of the same "narrow and bundle" idea; no single common policy type unifies them all (see Chapter 6).

Chapter 1. Survey Targets and System Overview

This chapter delimits the scope of the four repositories under survey and presents an overview of the layered architecture they form. Rather than understanding NBAccess, ClaudeRuntime, ClaudeOrchestrator, and SourceVault each as a standalone tool, they are easier to grasp as a single layered system that, starting from the Notebook, divides up LLM inference, workflow execution, knowledge storage, and auditing. The intent of this design is to place responsibilities closer to dangerous side effects and confidential information in the lower layers, and to let the upper layers concentrate on planning, coordination, and knowledge use, thereby creating a structure in which feature additions do not erode the safety boundaries. The four repositories do not implement the same functionality redundantly; they divide their roles along this gradient of responsibility.

Understanding current operation also requires looking beyond the four repositories and viewing claudecode as a background integration and compatibility layer. Fundamental functionality that has historically accumulated in claudecode—ClaudeEval, provider selection, Claude Directives, the shared polling tick, the palette, the MCP server registry, and more—still glues the four layers together today. In reading this chapter, it is therefore important to distinguish "the four layers as the logical architecture" from "claudecode as the implementation foundation during the transition period."

Nor is usage limited to one-question-one-answer ClaudeEval. Once deterministic routes via PromptRouter, workflow-ization of complex tasks via PromptWorkflow, the expansion from requirements to specification and implementation via spec-review/spec-impl, sharing with external LLMs via MCP, and autonomous triggering via AutoTrigger are all included, the system extends beyond a Notebook manipulation tool into a knowledge-and-execution foundation. Sections 1.1–1.3 below cover the survey targets and the logical architecture, 1.4–1.5 cover the implementation foundation of the transition period and the broadening of usage paths, and 1.6 summarizes the outlook.

1.1 Survey Targets

The repositories under survey are the following four.

- transreal/NBAccess
- transreal/ClaudeRuntime
- transreal/ClaudeOrchestrator
- transreal/SourceVault

As of the initial survey, each README stated the MIT License, and the default branch was main.

They differ greatly in scale and character. NBAccess is a comparatively small foundation package. ClaudeRuntime consists of a single-agent execution core augmented with external execution, process supervision, seat management, task placement, and the like. ClaudeOrchestrator is the upper control layer, encompassing workflows, observability, PromptWorkflow, and sessions. SourceVault is the largest repository, with many subpackages, spanning source management, search, web, mail, identity, mining, service management, and more.

1.2 Four-Layer Architecture

The basic dependency relationships can be represented as in Figure 1.

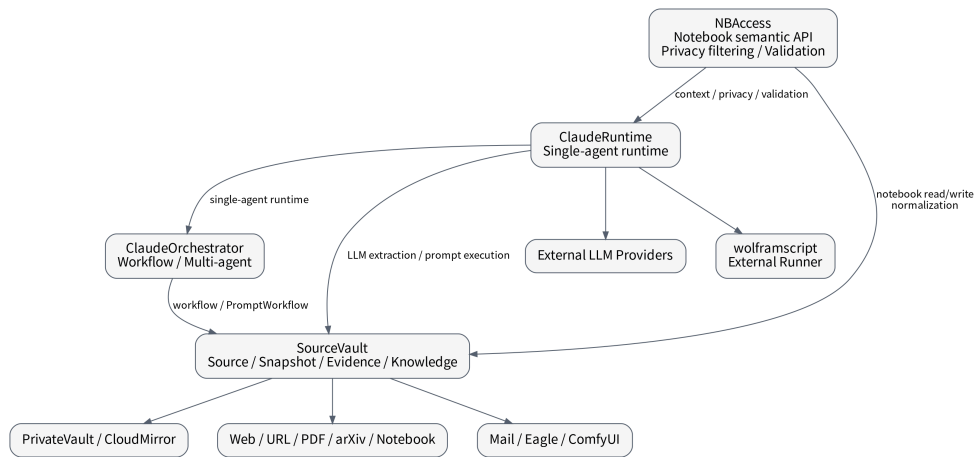


Figure 1. The four-layer architecture and its relationship to external systems

What matters in this figure is that ClaudeRuntime has no semantics of its own for Notebooks or secrets and calls NBAccess through adapters. Likewise, SourceVault manages the source lifecycle but delegates the safe execution of expressions to NBAccess and the coordination of multiple workers to ClaudeOrchestrator.

The actual system is not a simple one-way stack; it has a cyclic structure as shown in Figure 2.

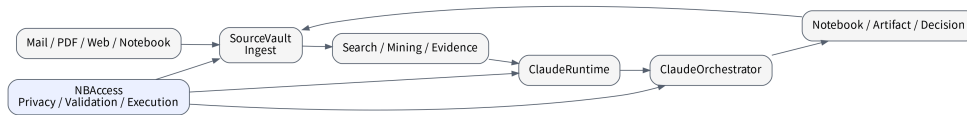


Figure 2. The knowledge-and-execution cycle centered on SourceVault

Evidence retrieved from SourceVault flows to the Runtime/Orchestrator, and the resulting Notebooks, code, reports, decisions, and workflow logs return to SourceVault. Work products thereby become inputs to subsequent exploration and inference.

1.3 Primary Anticipated Use Cases

The primary uses the four repositories anticipate fall into four broad categories.

The first is operating on confidential Notebooks while safely routing work between local and cloud LLMs.

The second is decomposing a complex task into multiple subtasks, running workers (serial by default; with configuration, DAG-parallel while preserving dependencies), verifying the artifacts, and committing them into a single Notebook.

The third is storing URLs, PDFs, arXiv papers, Notebooks, mail, and the like as immutable snapshots, and managing their relationships to claims, evidence, and artifacts.

The fourth is converting frequently recurring natural-language instructions into deterministic function calls or unevaluated expressions, making processing with the same meaning reproducible.

Even at this point, the design is optimized around Notebooks to a degree beyond typical RAG or agent frameworks, and the mining and safety layers described later further strengthen its character as a long-term knowledge foundation.

1.4 claudecode — The Integration and Compatibility Layer Behind the Scenes

Although the main subjects of this report are the four repositories, an accurate understanding of the current code cannot ignore transreal/claudecode. claudecode provides the integrated interface connecting Wolfram Language/Mathematica to the Claude Code CLI, the Codex CLI, the Anthropic/OpenAI APIs, LM Studio, and others, and retains a large body of fundamental functionality: ClaudeEval, provider/model selection, the Notebook-side UI, the shared polling tick, conditional injection of external package API documents, the MCP server registry for CLIs, and more.

Representative claudecode-side functionality	Relationship to the four layers
ClaudeEval / ContinueEval	The Notebook-side entry point into the Runtime's proposal loop and PromptRouter
<code>\$ClaudeModel</code> / <code>\$ClaudeAdvisaryModel</code> / <code>\$ClaudePrivateModel</code>	Selection of the normal, verification, and private models
Claude Directives	Projects rules and skills to Claude Code/Codex/workers
the <code>ClaudeRegisterPollingTick</code> family	The shared tick foundation for diagnostics, auto-trigger, and background job monitoring
<code>\$ClaudeCLIMCPServers</code>	A neutral registry that registers SourceVault MCP and others into headless CLI harnesses
palette / FE integration	The existing UI glue layer for specification generation/implementation, job progress, and writing results back

It is therefore appropriate to summarize the situation as follows: at the level of the logical architecture, the separation of responsibilities among NBAccess/Runtime/Orchestrator/SourceVault is canonical, while in actual operation during the implementation transition, claudecode functions as the "background glue layer" supporting bootstrap (package initialization, dependency loading, registry construction), UI, polling, and provider compatibility.

[Author's proposal] As an envisioned future cleanup, a natural configuration would separate the bloated claudecode.wl into core functionality and a compatibility-preserving legacy layer, and migrate to an operation centered on ClaudeOrchestrator / ClaudeRuntime / claudecode_core / NBAccess. However, as of July 16, 2026, it could not be confirmed in the public repositories that the claudecode_core separation has been implemented; this is not a settled roadmap but an organizing proposal by the author of this report.

1.5 Usage Has Expanded from "One Question, One Answer" to "Knowledge, Workflows, and Services"

Viewed from the user's perspective, the four layers offer at least five kinds of entry points. The first is interactive expression proposal via ClaudeEval. The second is deterministic high-level API invocation via PromptRouter. The third is long-running, multi-stage processing via ClaudeOrchestrator/SourceVault workflows. The fourth is use from external LLM clients via MCP, and the fifth is service-like autonomous processing via AutoTrigger, covering times when no Notebook is open.

Entry point	Typical examples	Executed by
ClaudeEval	"Summarize this Notebook," "Show my schedule"	claudecode → Runtime → NBAccess
PromptRouter	Saved Routes, UpcomingSchedule, FindNotebooks	SourceVault route → held expression
Workflow	Requirements→specification, specification→implementation, survey→verification→commit	Orchestrator / SourceVault workflow
MCP	Search from LM Studio, Directives lookup, artifact deposit	MCP service kernel + adapters
AutoTrigger	Launch workflows on schedule/event, dispatch to a specific PC	SourceVault scheduler + executor

Primary code and documentation references: [claudecode_info/docs/api.md](#) / [SourceVault_info/docs/user_manual.md](#) / [SourceVault_info/docs/api_workflowregistry.md](#)

1.6 Future Directions

The architectural focus going forward is to make the separation of responsibilities still more explicit and to clean up the transition-period glue code. In particular, if the functionality remaining in claudecode.wl can be divided into a permanent

core and a compatibility-preserving legacy portion, with the boundaries to ClaudeRuntime, ClaudeOrchestrator, NBAccess, and SourceVault fixed as API contracts, the system as a whole will become easier to understand and maintain.

Another focus is making the four layers observable not as "a collection of libraries" but as "a single operating model." If it becomes possible to trace, as a single run trace, which entry point a request came in through, which privacy decisions it passed, which LLMs and workflows it traversed, and which sv:// objects or Notebook cells the results returned to, then the separation of responsibilities in the design will also be reflected in operational observability.

In the long term, the natural direction is for the Notebook-centric interactive environment, headless services, MCP clients, and multi-PC workers to share a single policy, identity, and provenance model. This, however, is an [Integration goal]; at present, different policy vocabularies and registries coexist layer by layer (see Chapter 6). Even then, maintaining the principle that upper layers cannot expand the privileges of lower layers will be the key to preserving what makes this system distinctive.

Chapter 2. NAccess — The Final Boundary for Notebooks and Confidential Data

NAccess is the final boundary at the bottom of the four-layer stack, responsible for Notebook operations, confidentiality, access control, and safe expression execution. No matter how elaborate the plans built by the Runtime or the Orchestrator, it is NAccess that decides how much of a Notebook's confidential information may be exposed, whether a proposed Wolfram expression may actually be executed, and how secret keys are handled. The intent of this design is to consolidate the authoritative source of safety decisions in a single place and to structurally cut off any path by which upper-layer settings or convenience features could expand privileges. Understanding NAccess therefore explains why the upper layers are designed so that they cannot manipulate Notebooks freely.

Two mechanisms in this chapter are especially important. The first is that privacy is not attached only to the secret cell itself but propagates to variables and outputs derived from that cell. Even when a value is not disclosed directly, a result computed from a secret value can leak information, so before a Notebook context is handed to an LLM, dependencies are analyzed to decide whether to pass the body content or restrict it to schema-only. This idea also connects to SourceVault's privacy propagation and its MCP release gate. The second is that LLM output is treated not as a command but as a held expression. Proposed expressions are inspected before evaluation by `NBValidateHeldExpr` and related functions and classified into decisions such as `Permit`, `NeedsApproval`, `Deny`, and `RepairNeeded`. This also serves as a countermeasure against prompt injection and faulty generation, structurally eliminating any path where something is executed "because the LLM said so."

Sections 2.1–2.2 give an overview of the role and APIs; Sections 2.3–2.6 detail the individual mechanisms of privacy propagation, the file boundary, safe execution, and cryptography; Section 2.7 presents an assessment; and Section 2.8 summarizes future directions.

2.1 Role and Main Responsibilities

NAccess is the foundation layer that treats a Mathematica Notebook as an array of cells and provides cell-level reading and writing, privacy evaluation, dependency analysis, history management, and safe expression execution. Its central characteristic is that, when passing information from a Notebook to an LLM, it does not simply stringify the entire Notebook but evaluates each cell's privacy level and dependencies.

The privacy model distinguishes two values: `PrivacyLevel`, which expresses the confidentiality of the data itself (0.0–1.0; by default three levels are central: 0.0 = public, 0.75 = secret-dependent, 1.0 = secret), and `AccessLevel`, which expresses the ceiling of what a recipient may receive. Access permission is implemented as the `PrivacyLevel <= AccessLevel` test in `NBIsAccessible`; because the ceiling for the default privacy spec and for cloud providers (e.g., Anthropic) is `AccessLevel 0.5`, cells up to a `PrivacyLevel` of exactly 0.5 are allowed and cells exceeding 0.5 are excluded. Cells with high privacy, and outputs generated in dependence on them, are excluded from cloud-bound contexts.

Responsibility	Description
Notebook semantic access	Unified operations on cell retrieval, position, type, style, and content
Privacy filtering	Per-cell selection based on <code>PrivacyLevel</code> and <code>AccessLevel</code>
Dependency propagation	Propagation of confidentiality to outputs that depend on secret variables
Context projection	Generation of purpose-specific contexts such as full body, summary, and schema-only
Path/value normalization	Generation of object specs for SourceVault
History	Notebook change history and diffs
Safe execution	Validation, approval, and execution of held expressions
Crypto	KeyRef-based encryption, MACs, and key isolation

The key files are as follows.

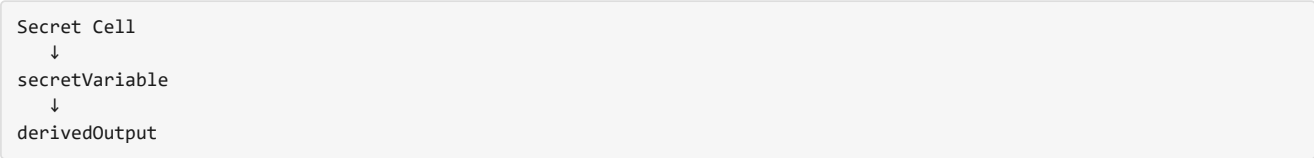
File	Main Role
NBAccess.wl	Notebook operations, privacy, dependency analysis, safe execution
NBAccess_crypto.wl	KeyRef-based encryption, MACs, key management
NBAccess_info/	Supplementary material such as design and API documents

2.2 Key APIs

API	Role
NBGetContext	Retrieve a Notebook context with the privacy filter applied
NBBuildGlobalVarDependencies	Build variable dependency relations
NBExtractAssignments	Extract assignment expressions
NBFileSpec	Convert a file into an object spec for SourceVault
NBNormalizePath	Normalize paths
NBValueSpec	Turn structural information about a value into an object spec
NBValidateHeldExpr	Determine the safety of a held expression
NBExecuteHeldExpr	Execute a permitted held expression
NBAuthorize	Policy-based execution authorization decision
NBGenerateSymmetricKeyRef	Generate a symmetric key and return a KeyRef
NBEncryptWithKeyRef	Encrypt using a KeyRef
NBVerifyMacWithKeyRef	Verify a MAC

2.3 Privacy and Dependency Propagation

Protecting secret information is not accomplished by excluding only the secret cell itself.



Even if the secret value is not written directly into derivedOutput, if it is a result computed from secretVariable, the secret information may still be inferable. NBAccess analyzes assignments, variable references, and output dependencies within the Notebook and propagates confidentiality. On that basis, it constructs a context appropriate to the destination and purpose.

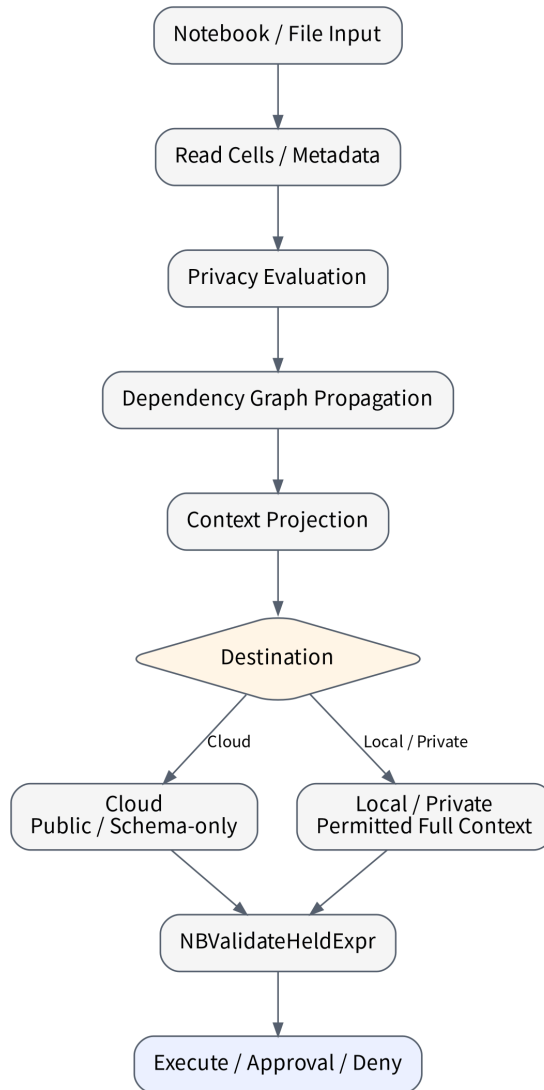


Figure 3. Privacy evaluation, context projection, and safe execution in NBAccess

For secret-dependent outputs, there is an option to send only the schema — type, size, keys, element count, and so on — without sending the body. This informs the LLM of the computational structure while withholding the values themselves.

2.4 File Privacy and the CloudPublishable Boundary

The behavior around NBFileSpec can be expressed conceptually as follows.


```

function NBFileSpec(path):
  if file does not exist:
    return Exists=False, PrivacyLevel=1.0

  ext = file extension

  if ext == ".nb":
    open notebook safely
    evaluate cell privacy
    count public/confidential cells
    propagate confidentiality through dependencies

    if notebook declares CloudPublishable=True:
      apply controlled public boundary
    if notebook declares CloudPublishable=False:
      PrivacyLevel = 1.0
  else:
    PrivacyLevel = file privacy policy result

  calculate content hash
  construct normalized path/value references
  cache base result
  reapply declared boundary after cache lookup
  return FileSpec

```

The Notebook-side declaration is stored in TaggingRules at {TaggingRules, "SourceVault", "CloudPublishable"} and can be read and written with NBGetCloudPublishable / NBSetCloudPublishable. What matters here is that the declaration is deliberately reapplied after the base spec's cache boundary, so that a stale cached privacy determination cannot mask the Notebook's latest declaration. The implementation comments state this explicitly: a stale base spec must never mask an explicit CloudPublishable declaration.

2.5 Safe Expression Execution

The design does not evaluate Wolfram Language expressions returned by the LLM directly with ToExpression or the like.

```

LLM Output
↓
HoldComplete[expression]
↓
NBValidateHeldExpr
↓
Permit / NeedsApproval / Deny / RepairNeeded
↓
NBExecuteHeldExpr

```

Dangerous heads, writes to Notebooks, external processes, credentials, network access, and the like are classified according to policy. Note that Permit / NeedsApproval / Deny / RepairNeeded is the classification vocabulary of the NBValidateHeldExpr family, whereas the higher-level NBAuthorize has a separate vocabulary: Permit / Deny / Screen / RequireApproval. The two play different roles, and care must be taken not to conflate them.

Because NBAccess holds this final decision, privileges cannot be expanded through Runtime- or Orchestrator-side settings alone.

2.6 Cryptography Layer

NBAccess_crypto.wl handles KeyRefs, not the keys themselves, in its external API. Conceptually it operates as follows.

```
function NBEncryptWithKeyRef(keyRef, plaintext):
  key = resolve internally
  if key type is not symmetric:
    fail

  encrypted = Encrypt(key, plaintext)

  if encryption failed:
    fail

  return serialized encrypted object
```

MAC verification uses a constant-time comparison rather than an ordinary string comparison. The implementation checks the lengths and then folds all bytes together with XOR, with no early return.

```
function NBVerifyMacWithKeyRef(keyRef, bytes, expectedMac):
  actualMac = calculate MAC using internal key
  return constant_time_equal(actualMac, expectedMac)
```

Key material is never emitted in return values or ordinary logs, and the KeyRef index holds metadata only. Two backends are defined: the default memory backend is intended mainly for development and testing (kernel-local only, not synchronized), while the SystemCredential backend is intended for production. The SystemCredential backend is [Implemented] through writing, reading, and deleting key material in the OS credential store, as well as persisting and reloading the metadata index. The remaining work is not "implementing the API" but maturing operational quality — specifically, propagation of backend write failures (the current `NBStoreCredentialKey` proceeds without inspecting the failure value from put), contention among multiple kernels, and cross-kernel regression testing. As for discrepancies between primary sources, `api_crypto.md` and some of the header comments in the source still contain the outdated statement that the write API is a TODO, and the return values of `NBStoreCredentialKey` / `NBDeleteCredentialKey` also differ between the API documentation (`keyRef / Boolean`) and the current code (`Association`) (see Appendix C).

2.7 Assessment

NBAccess is the clearest security boundary among the four repositories.

Its strength is that Notebook cells, variable dependencies, held expressions, safe execution, and cryptographic keys are designed as a single boundary.

At the same time, because a defect in this layer affects everything above it, it is also the package that requires the most rigorous regression tests, property tests, boundary-value tests, and cross-kernel tests.

2.8 Future Directions

For NBAccess going forward, the top priorities are formalizing the policy boundary and regression verification. In particular, confidential dependency propagation, the CloudPublishable boundary, held-expression bypass resistance, and the atomicity of Notebook semantic writes underpin the safety of everything above, so expanding property tests and cross-kernel tests in these areas is well worth the investment. In the cryptography layer, the immediate tasks for the implemented SystemCredential backend are verifying write-failure propagation, OS differences among credential stores, and multi-kernel contention, and completing the synchronization of the API documentation and return-value contracts.

In addition, the currently Notebook-centric privacy model needs to be applied consistently to `sv://` objects, MCP projections, and remote workflow results. NBAccess does not need to understand every medium directly, but if the same policy snapshot can be consulted at the boundaries of final Notebook writes, releases of contexts containing secrets, and external side effects, alignment with SourceVault's release decisions becomes considerably stronger.

In the long term, the desirable form is to fix NBAccess policy as a declarative `AccessSpec`, with the Runtime, the Orchestrator, and MCP all referring to the same digest. This would further reduce TOCTOU problems, where policy is swapped mid-execution, as well as policy drift across processes.

Chapter 3. ClaudeRuntime — The Expression-Proposal Loop

ClaudeRuntime is the layer that manages a single execution cycle: propose an expression from a prompt, then execute it after validation. Whereas NBAccess holds the final safety decisions, the Runtime is responsible for conversation state with the LLM, provider calls, proposal parsing, repair, and continuation decisions. In other words, the Runtime is not the authoritative source of safety; it is a progression-management engine that carries proposals to the safety boundary. The intent of this separation is to keep the skeleton of turn progression common even as providers, models, and execution methods change, and to make requests from any entry point ultimately converge on the same held-expression validation boundary.

What is worth understanding here is that behind what appears to be a single call to ClaudeEval lie the stages BuildContext, QueryProvider, ParseProposal, ValidateProposal, and DispatchDecision. Whether the PromptRouter returns a deterministic unevaluated expression or an ordinary LLM proposes an expression, everything is ultimately funneled into this proposal contract, which keeps the differences among entry points small.

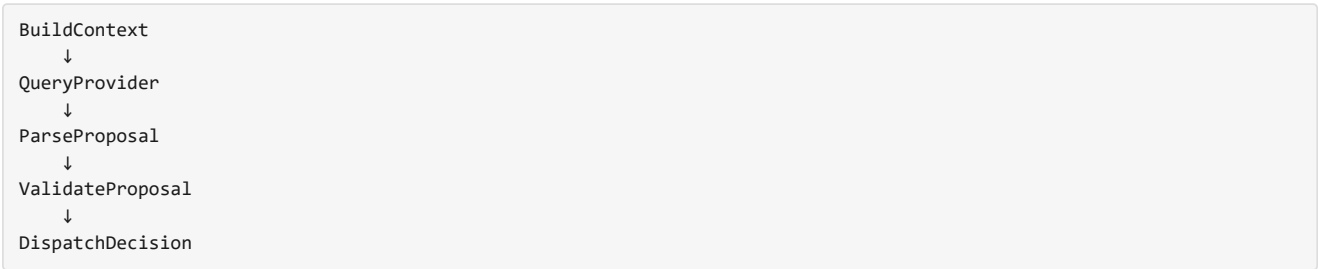
The Runtime additionally has an external runner, a process supervisor, a seat broker, and task placement, separating heavy processing from the main kernel. This makes it possible to run long processing without stalling the Notebook FrontEnd, but it also means handling operational failures of a different kind than LLM failures: PID reuse, license seats, cleanup failures, and the like.

Sections 3.1–3.4 describe the skeleton of turn execution; Section 3.5 covers the diversity of entry points and the boundary with the PromptRouter; Sections 3.6–3.7 discuss execution-location selection and external process management; Section 3.8 presents an assessment; and Section 3.9 summarizes future directions.

3.1 Role

ClaudeRuntime is an Expression-Proposal loop state machine that manages a single agent's one-off or continuing turns.

The basic flow consists of the following five stages.



The Runtime itself does not own the semantics of Notebooks, secret information, or access policy. Those are delegated to adapters and NBAccess. With this separation of responsibilities, the Runtime is in charge not of what may be executed, but of the order in which proposals are processed and the decision toward which they advance.

3.2 Key Files

File	Main Role
ClaudeRuntime.wl	RuntimeState, turn DAG, proposal loop
ClaudeRuntime_externalrunner.wl	External execution via wolframscript
ClaudeRuntime_processsupervisor.wl	External process supervision
ClaudeRuntime_seatbroker.wl	Execution seat management
ClaudeRuntime_session.wl	Runtime session
ClaudeRuntime_sessionrunner.wl	Session execution support

File	Main Role
ClaudeRuntime_taskplacement.wl	Task classification and backend recommendation
ClaudeRuntime_info/	Design and API documents

3.3 Adapter Model

When a Runtime is created, an adapter supplies the required capabilities. The contract for a multi-turn adapter (CreateClaudeRuntime) consists of the following seven functions.

```
BuildContext
QueryProvider
ParseProposal
ValidateProposal
ExecuteProposal
RedactResult
ShouldContinue
```

This adapter structure separates the LLM provider, CLI, local models, Notebook context, and execution method from the Runtime core. Note that the single-turn Transition execution API called from Orchestrator workflows, `ClaudeRuntimeExecuteTransition[adapter, contextPacket]`, is a reduced contract requiring only five of these — `BuildContext`, `QueryProvider`, `ValidateProposal`, `ExecuteProposal`, and `RedactResult` — and its scope of application differs from the public contract of the multi-turn adapter.

3.4 Turn Execution and the Decision Model

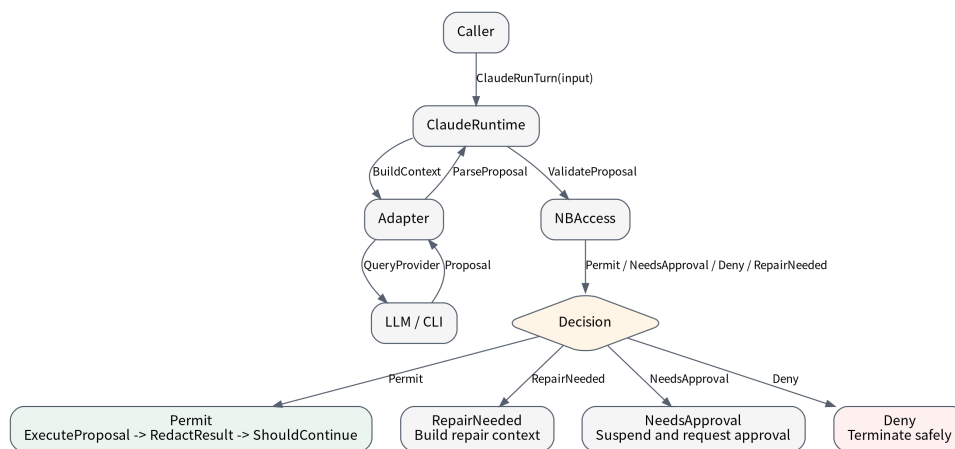


Figure 4. The Expression-Proposal loop in ClaudeRuntime

ClaudeRunTurn builds nodes such as `buildContext`, `queryProvider`, `parseProposal`, `validateProposal`, and `dispatchDecision` into a DAG and launches the job. The core point is that the LLM response is never evaluated directly; it is parsed as a proposal, and validation by NBAccess is mandatory.

Decisions need to be understood in two tiers. First, the validation decisions returned by the adapter's `ValidateProposal` contract (NBAccess validation) are the following four.

Validation decision	Meaning
Permit	Validated and executable
NeedsApproval	Requires user approval
Deny	Not executable
RepairNeeded	Proposal requires repair

Second, the Runtime may examine the shape of the proposal itself and produce a dispatch outcome before proceeding to validation.

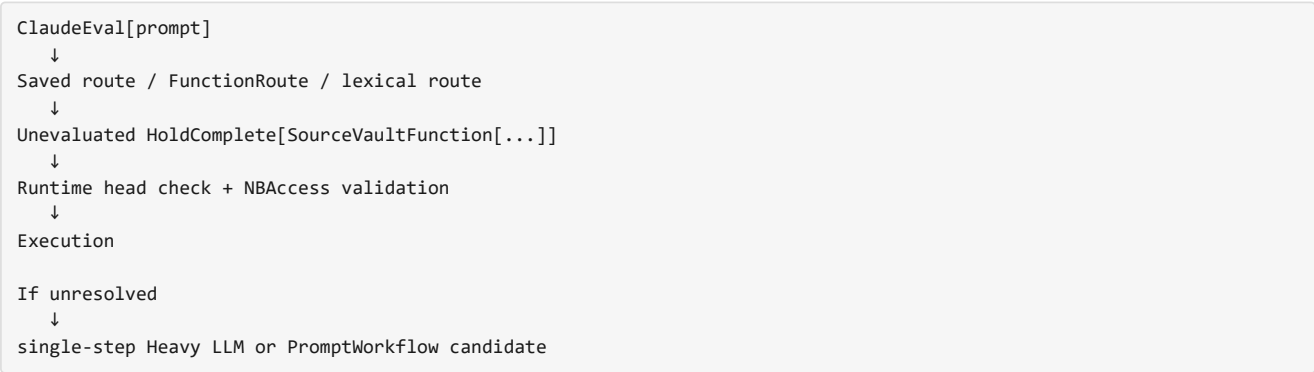
Runtime dispatch outcome	Meaning
TextOnly	Handled as a textual answer rather than expression execution
ToolUse	Proceeds along the explicit tool path

Note that TextOnly and ToolUse are not classifications of NBAccess validation; they are dispatch decisions produced by the Runtime. This two-tier classification makes it possible to branch non-executable responses into textual answers, repair, pending approval, and so on, rather than treating them all as errors. Promotion to RepairNeeded on contract violations and the escalation logic from Permit to NeedsApproval are also implemented.

3.5 Diversity of Entry Points — The Boundary Between ClaudeEval and the PromptRouter

A point to keep in mind when trying to understand ClaudeRuntime is that ClaudeEval is not a function that always lets an LLM generate freely. In the current Notebook-side entry point, a saved PromptRoute or a deterministic SourceVault route can be proposed before any LLM is called.

SourceVaultProposePromptRoute returns not an evaluated result but an unevaluated expression wrapped in HoldComplete[...]; only that expression is passed to the Runtime and is executed after passing the head allowlist and NBAccess validation. For example, the routine request "my schedule for the next three days starting today" can be converted into a frozen function call to SourceVaultUpcomingSchedule instead of having a heavyweight LLM interpret it every time. Likewise, when a saved prompt matches exactly, SourceVaultProposeSavedPromptRoute can present the existing route before any LLM call. This allows the same request to be re-executed at low cost and with high reproducibility while preserving ClaudeEval's safety contract of pre-evaluation validation.

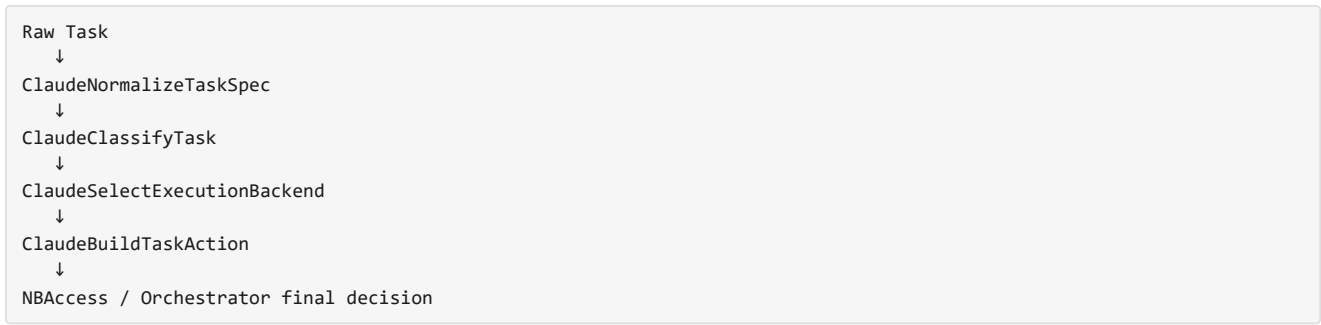


To lay out the responsibility boundary: the PromptRouter resolves which high-level function to route to, but it is not an authority holding execution rights over expressions. The Runtime, for its part, manages turn progression including dialogue with the provider, proposal parsing, and repair/approval, but owns neither Notebook privacy nor the final execution authority. Thanks to this separation, whether an expression was constructed deterministically by the PromptRouter or freely generated by an LLM, it ultimately converges on the same held-expression validation boundary.

Primary code and documentation references: [SourceVault_info/docs/api_promptrouter.md](#) / [SourceVault_info/docs/user_manual.md](#) / [claudecode_info/docs/api.md](#)

3.6 Task Placement

ClaudeRuntime_taskplacement.wl makes advisory determinations of where a task should run.



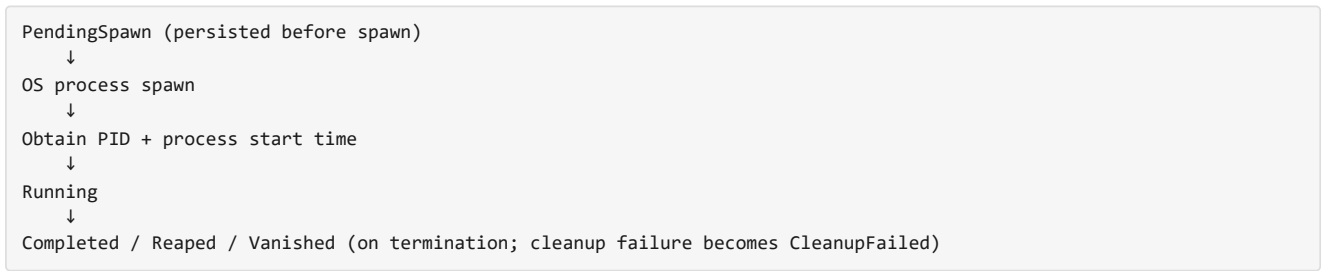
The typical execution targets can be organized as follows.

Execution Target	Suitable Workloads
Main Kernel	Notebook UI, FrontEnd-dependent work, live state manipulation
Subkernel	Side-effect-free numerical computation, parallel pure computation
WolframScript	Long-running batches, external I/O, bulk ingest, independent processes

Final permission is determined by NBAccess policy and the Orchestrator's workflow policy. A Runtime recommendation alone never moves execution to a dangerous backend.

3.7 External Process Management

The external runner separates long-running processing from the main kernel. The Process Supervisor uses a two-phase manifest.



Recording the process start time in addition to the PID reduces the risk of mistakenly killing a different process when the OS reuses a PID.

Cleanup at termination follows a fixed order — seat release, then archive — and if seat release fails, the entry is left in place as CleanupFailed without archiving, to be targeted for diagnostics. Archive is one step of the cleanup procedure (and a storage directory), not an independent state name.

3.8 Assessment

ClaudeRuntime's greatest strength is its clear single responsibility. Because it delegates safety decisions to NBAccess, the coordination of multiple agents to the Orchestrator, and long-term knowledge to SourceVault, the Runtime core can concentrate on the proposal loop.

On the other hand, the relationships among adapters, legacy claudecode, the external runner, sessions, and task placement are complex, and load order, compatibility modes, and fallback paths must be managed explicitly.

3.9 Future Directions

For the Runtime going forward, it will be important to organize RuntimeSession as a public lifecycle, beyond just one-off turns. The current implementation has ConversationState, TurnCount, a tool loop, and checkpoint building blocks, but

durable checkpoints that survive process restarts, budget suspend/resume, and per-tool-call approval have not yet been integrated into a complete session protocol.

From the standpoint of prompt-injection defense, it is also important to make PreparedInputToken verification immediately before sending to the LLM fully enforceable at the Runtime/adaptor boundary. By pinning down which messages, tool schemas, and retrieval refs are sent to which provider across the entire request envelope, and verifying this immediately before transmission, substitution after context construction can be prevented.

As for task placement, there is room to extend it beyond performance optimization into an execution contract that covers privacy, license seats, GPU/CPU resources, and SpecificMachine constraints. Connected with the Orchestrator's resource place, this can evolve naturally from execution-location selection for a single turn into resource scheduling across multiple PCs.

Chapter 4. ClaudeOrchestrator — Multi-Agent Orchestration and the Single Committer

ClaudeOrchestrator is the layer that decomposes complex requests into multiple workers, integrates their deliverables, and carries the work through to the final commit. It is easiest to understand as the flow Planning, Spawn, Reduce, Commit. The Planner decomposes the work into TaskSpecs, workers produce individual artifacts, the Reducer integrates them, and finally only the single committer applies the result to live state such as Notebooks.

The core of this layer is the single-committer principle, which is not a mere implementation convenience but central to safety and reproducibility. If workers write directly to the same Notebook, conflicts, order dependence, partial failures, and difficulty of rollback arise. Workers are therefore restricted to being artifact producers, and shared items are passed via SourceVault or explicit artifact refs. MCP deposit and `sv://artifact/...` become natural media for sharing deliverables between agents while maintaining this principle.

Furthermore, because the Orchestrator uses Petri net workflows, it can handle what is hard to express with a simple DAG alone: waiting for approval, retries, joins of multiple branches, waiting for external job completion, and cancel/recovery. PromptWorkflow can convert complex natural-language requests into workflow candidates, and in spec-review/spec-impl the development process itself — "create a specification from requirements, then implement and verify that specification" — becomes a workflow.

In what follows, Sections 4.1–4.4 describe the role and the basic structure of worker governance, Section 4.5 describes each stage of the execution pipeline, Sections 4.6–4.8 cover Petri nets, PromptWorkflow, and turning development into workflows, Section 4.9 gives an assessment, and Section 4.10 summarizes future directions.

4.1 Role

ClaudeOrchestrator is a multi-agent orchestration layer that organizes multiple Runtimes or workers from above.

The basic flow consists of the following four stages.

```
Planning
↓
Spawn
↓
Reduce
↓
Commit
```

Whereas ClaudeRuntime handles a single proposal loop, ClaudeOrchestrator manages task decomposition, dependencies, workers, deliverables, and commit ordering.

4.2 The Core Design Shift — Separating Workers from the Committer

In past designs where workers wrote directly to the live Notebook, there were problems such as inconsistency of shared state across multiple workers, unstable resolution of `EvaluationNotebook[]`, unintended execution of `CreateNotebook`, indeterminate commit ordering, and difficulty of rollback.

The design therefore moved to one in which workers are restricted to being artifact producers and only the single committer is permitted to apply changes to the live Notebook.

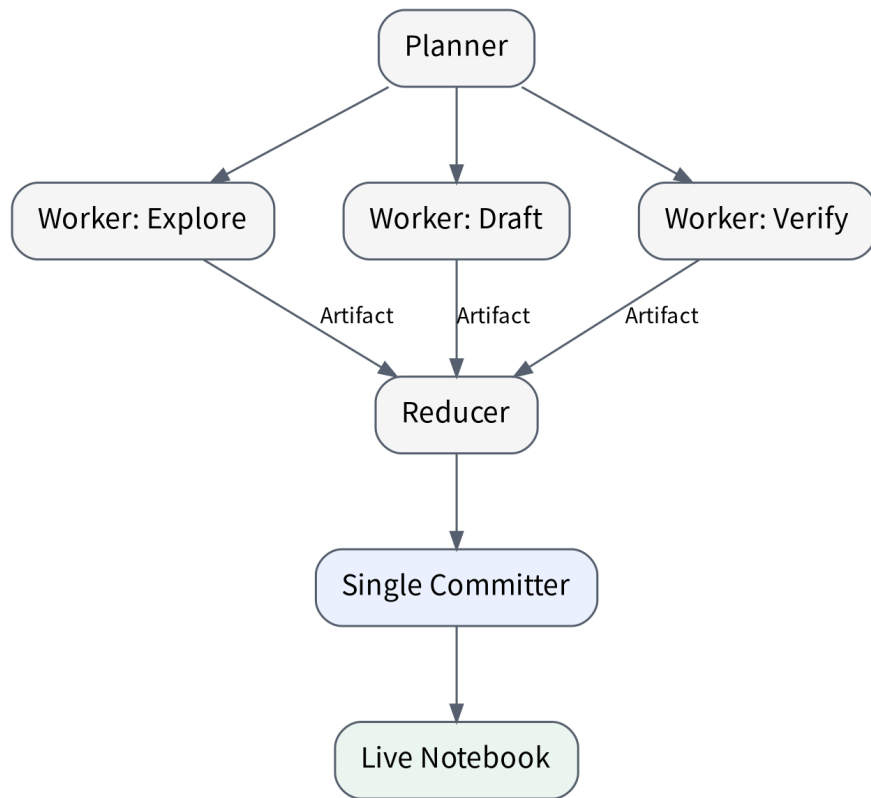


Figure 5. The worker/reducer/single-committer structure of ClaudeOrchestrator

What is shared between workers is not the Notebook itself but artifacts, JSON, Associations, and explicit references.

4.3 Key Files

File	Main Role
ClaudeOrchestrator.wl	roles, capabilities, planner, spawn, reduce, commit
ClaudeOrchestrator_workflow.wl	multi-token Petri net
ClaudeOrchestrator_promptworkflow.wl	Turning complex prompts into workflows
ClaudeOrchestrator_observability.wl	Observation of LLM calls, handlers, and workflows
ClaudeOrchestrator_session.wl	Session support
ClaudeOrchestrator_info/	Design, API, and examples

4.4 Roles and Capabilities

Representative roles are Explore, Plan, Draft, Verify, Reduce, and Commit.

Each role is assigned capabilities, and worker roles are not granted permission to write directly to Notebooks.

Representative deny targets include NotebookWrite, CreateNotebook, EvaluationNotebook, RunProcess, SystemCredential, unrestricted URLRead, and direct side effects on live resources.

4.5 Execution Pipeline — Planning, Spawn, Reduce, Commit

4.5.1 Planning

ClaudePlanTasks decomposes a parent task into a TaskSpec DAG. A TaskSpec has the following fields.

```
TaskId
Role
Goal
Inputs
Outputs
Capabilities
DependsOn
ExpectedArtifactType
OutputSchema
```

The Planner's output is not executed as-is; its schema and dependencies are validated.

4.5.2 Spawn

ClaudeSpawnWorkers launches workers while respecting dependencies and collects their deliverables.

```
Task A: Explore
Task B: Draft, depends on A
Task C: Verify, depends on B
```

Each worker receives only its own TaskSpec, permitted inputs, capabilities, and output schema. Results are aggregated into Artifacts: taskId -> artifact and Failures.

The default execution order is serial execution with `MaxParallelism -> 1`; specifying `MaxParallelism > 1` OR `UseDAG -> True` switches to a DAG-parallel path that preserves dependencies [Implemented]. Note that, as one of the discrepancies between primary sources, the API documentation still contains the outdated statement "serial/sequential, currently 1" (see Appendix C).

4.5.3 Reduce

ClaudeReduceArtifacts integrates multiple artifacts. Reduction can use deterministic processing or LLM-based integration, but it must preserve source references, task references, and verification state.

The Reducer does not write to the live Notebook; it produces a ReducedArtifact.

4.5.4 Commit

ClaudeCommitArtifacts performs the final application as the single committer. The commit path is selected by the `CommitMode` option, and the default is "Direct" (direct application through an approved path). The following diagram shows the path when `CommitMode -> "Transactional"`, which includes a shadow buffer, verification, and rollback, is selected.

```
Reduced Artifact
  ↓
Schema validation
  ↓
Shadow buffer
  ↓
Verifier
  ↓
Commit plan
  ↓
NotebookWrite through approved path
  ↓
Post-commit verification
  ↓
Rollback if required
```

In Transactional mode, erroneous worker output is prevented from directly corrupting the Notebook, and on failure the target can be kept unchanged. The default Direct mode has no such shadow buffer/rollback stage, so when evaluating the accident tolerance of commits, one must distinguish which mode is in use.

4.6 Petri Net Workflow

ClaudeOrchestrator_workflow.wl provides a multi-token Petri net that is more general than a simple DAG. Petri nets make it easy to express transitions that fire only when multiple conditions are satisfied, multiple tokens, retries, waiting for approval, waiting for external job completion, branch joins, cancellation, timeouts, repair loops, and more.

4.7 PromptWorkflow — Converting Complex Natural-Language Requests into Workflow Candidates

ClaudeOrchestrator_promptworkflow.wl is the layer that converts complex prompts that are hard to express as a single-function route into WorkflowNet candidates. It also has a classifier that routes short factual questions and simple function queries back to the Runtime's single path.

Here too, a new workflow is never executed the moment the LLM outputs it. First, if an existing FunctionRoute/WorkflowRoute matches uniquely, it is preferred and new generation is avoided. If no existing route matches, a local complexity assessment is performed before calling the LLM. A prompt becomes a WorkflowCandidate when there is an explicit "turn this into a workflow" request, multiple subtask verbs, or task verbs coexisting with control words such as "first/next/finally".

When a prompt becomes a candidate, ClaudeProposeWorkflowNetFromPrompt proposes WorkflowNet code, but only after a static scan for forbidden patterns, a held parse, and extraction of the declarative structure. A newly generated workflow stops at WorkflowRouteDraft's NeedsApproval — even if the prompt says "create it and run it immediately" — and is neither auto-registered nor auto-executed. This is an important boundary separating "the LLM's ability to generate code" from "the authority to activate a workflow as an authority."

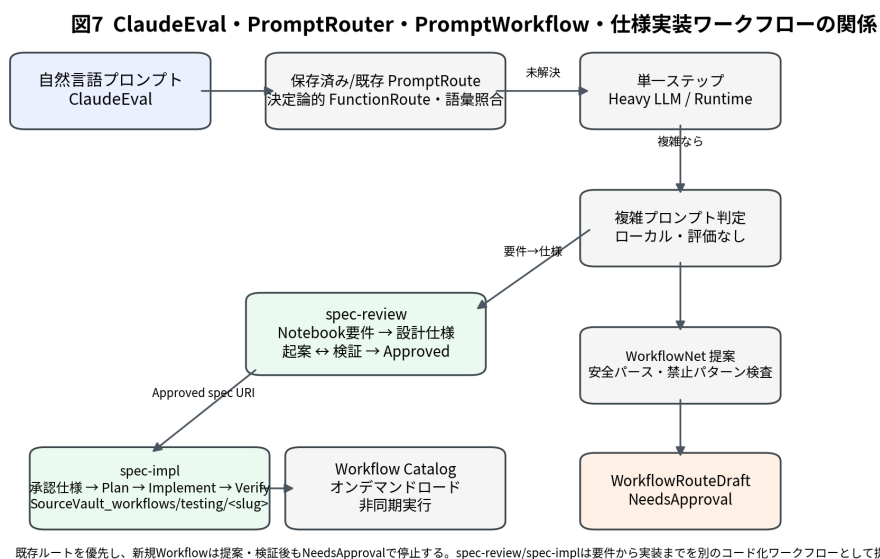


Figure 6. Relationships among ClaudeEval, PromptRouter, PromptWorkflow, and the specification-implementation workflows

4.8 Turning the Process from Generating Specifications from Requirements to Implementation into Workflows

Separately from PromptWorkflow, SourceVault_workflows contains spec-review and spec-impl as system workflows that automate software development itself in two stages. spec-review takes requirements text in a Notebook as input and, through back-and-forth between the drafting model (`$ClaudeModel1`) and the verifying model (`$ClaudeAdvisaryModel1`, default Codex), converges a Wolfram design specification Markdown to Approved. The approved specification and reviews are saved as SourceVault immutable snapshots, and the approved specification can be referenced at `sv://snapshot/OrchSpec/...`

spec-impl receives the approved specification, iterates Plan → optional plan review → Implement → Verify, and generates a loadable `SVWorkflow_<Canon>` package under `SourceVault_workflows/testing/<slug>/`. Verification does not rely on LLM review alone; it has multiple stages: a static smoke check, a hard gate consisting of dynamic load+launch in a fresh wolframscript, execution of generated tests, and finally LLM verification of conformance to the specification. Newly generated products always enter the testing stage and are promoted to production after testing.

Stage	Input	Main Processing	Artifacts in SourceVault
spec-review	Notebook requirements	Iterate drafting ↔ verification until Approved	<code>orch/<project>/{requirements,spec,review}; sv://snapshot/OrchSpec/...</code>
spec-impl	Approved design specification	Plan → Implement → Verify	<code>impl/<slug>/{plan,planreview,artifact,verify}</code>
Catalog registration	Verified workflow package	Generated into the testing stage, on-demand load	<code>SourceVault_workflows/testing/<slug>/</code>

Heavy LLM loops are run in a background wolframscript so as not to freeze the Front End kernel, and the FE side monitors progress/result via a shared polling tick. Even on failure, the intermediate plan, artifacts, and verify chain are left in SourceVault, so this is not mere code generation: the ability to reconstruct the design process and the verification process is a characteristically SourceVault trait.

```
Primary code and documentation references: ClaudeOrchestrator_promptworkflow.wl / SourceVault_workflows/spec-review/SVWorkflow_SpecReview_info/design/implementation-spec.md / SourceVault_workflows/spec-impl/SVWorkflow_SpecImpl_info/design/implementation-spec.md / SourceVault_info/docs/api_workflowregistry.md
```

4.9 Assessment

ClaudeOrchestrator's strongest design decision is the separation of workers and the committer. In typical multi-agent systems, each agent may call tools directly and impose side effects on the shared environment. Here, by restricting workers to being artifact producers and making commit an explicit, single responsibility, Notebook consistency and auditability are improved.

4.10 Future Directions

Going forward, it is important to clarify the boundary between RuntimeSession and the Petri net. Expanding every model turn or ordinary tool call into Petri tokens causes state explosion, so the appropriate direction is for the Orchestrator to manage only episode start, approval, budget, and artifact commit, while confining the internal turn/tool loop to the Runtime.

For sharing between agents, the design that uses SourceVault as an artifact bus becomes even more important [Partially implemented / integration goal]. In a configuration where worker B searches for and retrieves the `sv://artifact/...` that worker A deposited via MCP, and the Reducer integrates them while preserving reference relationships, the Notebook need not be abused as shared memory. The deposit/search/get APIs are each implemented, but not all inter-worker communication has been unified onto this path.

In the long term, the goal is to integrate the workflow catalog, AutoTrigger, and multi-PC placement so that the same workflow can be launched consistently from manual invocation, the PromptRouter, schedules, SourceVault events, and remote PCs. In doing so, approval, stage promotion, and diagnostics gates should be kept between automatic workflow generation and automatic execution.

Chapter 5. Foundational Architecture of SourceVault

SourceVault is designed not as a mere document-search database but as a source-first knowledge vault that tracks "what was used as evidence, which version was used, and what was generated." URLs, PDFs, arXiv papers, Notebooks, and mail are registered as sources; their content at that point in time is stored as immutable snapshots; and summaries, claims, search chunks, and artifacts are separated out as derived artifacts. The aim of this design is to be able to trace the scope of impact when upstream material changes. Even if a URL stays the same, updated content becomes a different snapshot, and Evidence Bundles or generated outputs grounded in an old snapshot can be treated as stale candidates. The central value of SourceVault therefore lies not in search speed but in source identity, version, provenance, and re-executability.

For exchange with the outside, the `sv://` URI becomes important. It expresses objects, snapshots, artifacts, chunks, and so on as stable opaque references, so that the same target can be referred to across MCP and workflows. This makes it possible to build a structure in which, instead of passing a huge body of text directly to an LLM, the necessary projection is retrieved when needed, behind a release gate. The gateway that opens this URI layer to external LLM clients is SourceVault MCP, and its significance lies in being designed as a projection layer rather than an authority.

In what follows, Sections 5.1–5.4 describe the source-first design and lifecycle management, Sections 5.5–5.6 the deterministic usage paths and the release gate for search, and Sections 5.7–5.9 the `sv://` URI, MCP, and external LLM integration; Section 5.10 gives an assessment and Section 5.11 summarizes future directions.

5.1 Source-First Design — Source, Snapshot, and Derived Artifact

SourceVault is the largest of the four repositories and has the broadest functional scope. Its basic philosophy is to canonicalize the identity, version, fetch time, content hash, and provenance of the source itself before any LLM-generated summaries or answers. Target sources include URLs, web pages, PDFs, arXiv papers, Notebooks, text, mail, Eagle-managed materials, generated artifacts, and workflow results.

SourceVault separates the logical source from the snapshot, which represents its content at a point in time.

```
Source
├─ Snapshot v1
├─ Snapshot v2
└─ Snapshot v3
```

Even the same URL or paper may have its content updated over time. A Source represents identity, while a Snapshot represents an immutable version based on the content hash.

Summaries, claims, embeddings, search indexes, and generated code are treated as derived artifacts, separate from the original source.

```
Snapshot
├─ Parsed Text
├─ Claims
├─ Summary
├─ Search Chunks
└─ Evidence Bundle
```

This separation makes it possible to determine which derived artifacts have become outdated when the original material is updated.

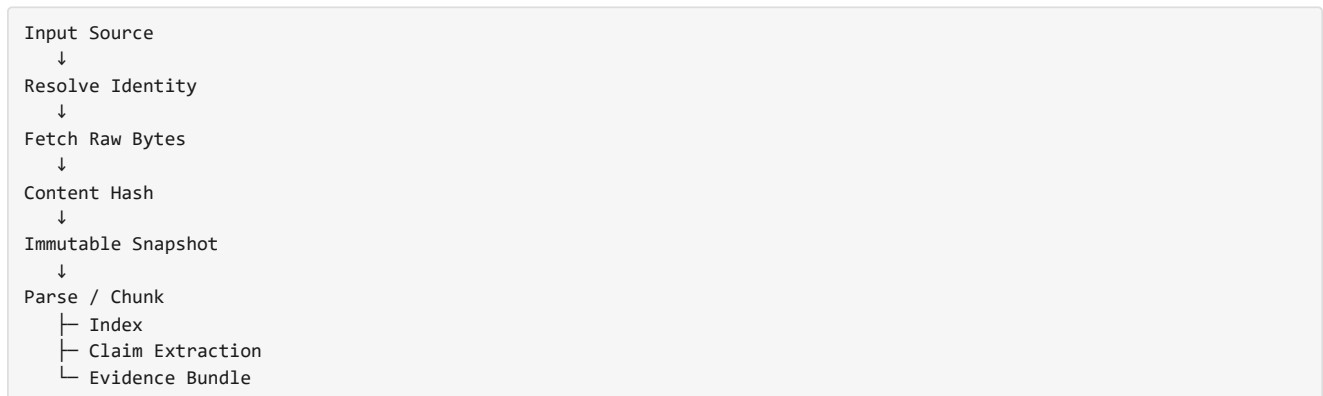
5.2 Module Structure

File	Main role
SourceVault.wl	umbrella loader and integrated entry point
SourceVault_core.wl	immutable snapshots, events, vault core

File	Main role
SourceVault_contracts.wl	FunctionContract, failure schema, initialization DAG
SourceVault_promptrouter.wl	PromptRouter, saved prompts, privacy routing
SourceVault_searchindex.wl	release context and search index
SourceVault_searchview.wl	working surface for navigating search results
SourceVault_servicemanager.wl	service, jobs, heartbeat
SourceVault_webingest.wl	Web/PDF fetching and parsing
SourceVault_identity.wl	Identifier/Entity
SourceVault_mining.wl	Tag/Authorship/MiningObject
SourceVault_oopsseed.wl	OOPS seed ontology
SourceVault_mailstructure.wl	structuring of general mail
SourceVault_knowledgehome.wl	Knowledge Home
SourceVault_diagnostics.wl	SystemDoctor
SourceVault_capbroker.wl	CapabilityLease/PreparedInputToken
SourceVault_taint.wl	input trust and taint
SourceVault_cognition.wl	cognitive support
SourceVault_adjudication.wl	multi-LLM adjudication
SourceVault_anomaly.wl	observe-only anomaly analysis

Because the number of functions is large, SourceVault has a layer that contractualizes return values, failures, and initialization dependencies (FunctionContract in SourceVault_contracts.wl). By using a standardized Failure schema, it can distinguish the cause, a publicly exposable reason, and internal diagnostics, rather than a mere `$Failed`.

5.3 Ingest Flow



Raw data is stored per content hash, and metadata references the source and snapshot. Even when the same content is fetched from a different URL, duplicates can be handled via the content hash.

5.4 Claims and Evidence Bundles

A Claim is an assertion extracted from a source.

```
Claim
├─ Claim Text
├─ Source Ref
├─ Snapshot Ref
├─ Span Ref
├─ Confidence
└─ Extractor / Run
```

Claims do not replace the original source; they are managed as append-only derived records.

An Evidence Bundle collects the sources, snapshots, claims, and generated files that a deliverable depends on.

```
Evidence Bundle
├─ Sources
├─ Snapshots
├─ Claims
├─ Generated Files
└─ Generator / Workflow
```

When an upstream snapshot becomes Stale or Invalidated, bundles that reference that snapshot also become subject to re-evaluation.

5.5 PromptRouter and Deterministic Usage Paths

PromptRouter converts a natural-language prompt into a deterministic function call or a workflow proposal. The important point is that the Router itself does not evaluate expressions.

```
Natural Language Prompt
↓
Route Proposal
↓
Held / Unevaluated Expression
↓
NBAccess / Runtime Validation
↓
Execution
```

When ambiguous, it does not execute on its own; it returns candidates or NeedsApproval.

From the user's perspective, SourceVault does not start out as a "vector store for LLMs"; instead, the identity and version of sources are registered first. The basic operations are ingest of URLs/PDFs/text/arXiv via SourceVaultIngest, Notebook indexing via SourceVaultIndexNotebook, and status checks via SourceVaultStatus/ListSources/ListSnapshots. A re-ingest with the same content hash does not create duplicates, and the stable identity of a source is separated from its per-point-in-time snapshots.

Usage paths split into deterministic paths and LLM paths. Ingest, Notebook indexing, lint, FindNotebooks, known PromptRoutes, and the like can operate without an LLM. Processing that requires semantic understanding, such as structured claim extraction and Notebook summaries, uses an LLM via ClaudeRuntime. This separation keeps source management itself from depending on any particular LLM provider.

Usage scenario	Representative operations	LLM dependency
Material registration	SourceVaultIngest[URL/PDF/text/arXiv]	Not required
Notebook management	SourceVaultIndexNotebook, Header/ToDo/Lint	Not required
Evidence reference	Span/Context, snapshots, Evidence Bundles	Not required
High-level queries	UpcomingSchedule etc. via PromptRouter	Depends on the route; deterministic routes require none
Semantic extraction	SourceVaultExtract with an LLM schema, Notebook summaries	Via ClaudeRuntime

Usage scenario	Representative operations	LLM dependency
Complex work	Orchestrator workflows, spec-review/spec-impl	Multiple LLMs can be used
External LLM integration	MCP search/get/directives/deposit	Depends on the client/model access profile

5.6 Release Context and Fail-Closed Search

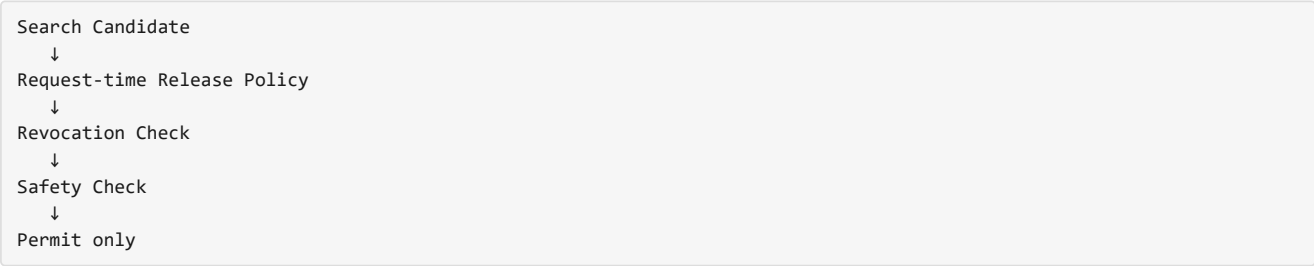
SourceVault search requires a ReleaseContext. The values involved in gate decisions need to be understood as split between the policy side and the data side.

First, the policy values of a registered ReleaseContext are as follows.

```
MaxPrivacyLevel    (Required. The maximum privacy that may be returned)
RequiredTags       (Tags required to be present; allow-list check)
DenyTags          (Tags that cause rejection if present)
ReleaseContextTag / Sink / DisplayName / RequireCitation /
AllowAnswerGeneration etc. (boolean defaults are the safe-side False)
```

On the other hand, the metadata held on the source/chunk side subject to release includes PrivacyLevel, Tags, State (only Approved / Published / Released pass the gate), and ValidFrom / ValidUntil (validity period), and the gate makes its decision by matching the policy values against these. Note that State and ValidFrom/ValidUntil are not fields of the ReleaseContext itself.

Search results are re-evaluated not only at build time but also by a request-time gate immediately before being returned, and if the build-time RevocationEpoch has expired, the return is refused. Note that the design specification describes the decision in terms of a concept called "GateContext," which composes ReleaseContext, AccessRequest, and RevocationEpoch, but this is not implemented as a single public API type [Design-only]. The release policy in searchindex and the AccessRequest/AccessGrant and RevocationEpoch on the MCP side are separate public APIs.



Unregistered or incomplete contexts result, fail-closed, in empty results or rejection rather than broad permission. Note that TrustDomain (the cloud/local destination classification) used in LLM routing and the Purpose of a CapabilityLease are concepts belonging to PromptRouter and the Capability Broker respectively, and are not fields of ReleaseContext. The same kind of "narrow the purpose" idea is implemented independently at each layer.

5.7 sv:// URI — Canonical Identity Used Across SourceVault

The SourceVault Universal MCP Access specification establishes the policy of canonicalizing references exchanged among SourceVault/MCP/Web/Orchestrator into sv://... URIs. The important point is that data classes such as mail, web, and pdf are not embedded in the URI. The medium type is placed in sidecar metadata called Class/MediaType/Kind, and the URI is kept strictly as an opaque identity.

The namespaces that specification v2 canonically reserves as the first segment are nine: object, chunk, artifact, hash, group, relation, snapshot, record, and citation. The current URI parser (\$SourceVaultURINamespaces) reserves three more — file, packageapi, and directive — accepting 12 namespaces in total. Note, however, that "reserved in the parser," "in use as tool output," and "wired as a data adapter" are distinct states.

Namespace example	Purpose	Status
<code>sv://object/<id></code>	general object identity	Canonically reserved in v2
<code>sv://chunk/<id></code>	chunk for search/grounding	Canonically reserved in v2
<code>sv://artifact/<id></code>	artifact generated by an LLM or workflow	Canonically reserved in v2
<code>sv://hash/sha256/<hex></code>	content-addressed hash	Canonically reserved in v2
<code>sv://snapshot/<class>/<hex></code>	immutable snapshot	Canonically reserved in v2
<code>sv://group/... / relation / citation / record</code>	relations, groups, citations, records	Canonically reserved in v2
<code>sv://file/<id></code>	allow-listed file reference (Directives are also projected here)	Reserved in the current parser (an implementation extension outside the v2 reservation). Adapter wired
<code>sv://packageapi/<id></code>	package API contract/document	Reserved in the current parser (an implementation extension outside the v2 reservation). Adapter wired
<code>sv://directive/<id></code>	stable opaque id for directives	Reserved in the current parser. An independent adapter is not yet wired; the current output of the <code>sourcevault_directives</code> tool is <code>sv://file/...</code>

In other words, for directives, the reservation in the URI grammar is implemented, but the full migration to the independent directive adapter defined by the retrieval specification is incomplete, and the current tool output goes through the file namespace [Partially implemented].

SourceVaultCanonicalURI is the normalization point before a URI is stored as a key, edge, group member, or SourceRef. The legacy `blob:sha256:...` and `snapshot:class:...` forms are accepted as compatibility inputs and can be canonicalized to `sv://hash/...` and `sv://snapshot/...`. Therefore, rather than saying "all internal records inside SourceVault are already stored solely as URIs," it is more accurate to say that the stable common reference layer for external APIs, search, and relation descriptions has been unified onto `sv://`. Because a URI is an identity and not a capability, knowing a URI does not confer read permission.

5.8 SourceVault MCP — A Gateway for External LLMs as a Projection Layer

There is more than one way to let an LLM use SourceVault. `PromptDeliveryProfile` looks at whether the model and its execution environment can use MCP, whether they can read a local folder, and whether SourceVault input is fully mediated, and selects one of the strategies `MCPDeferred`, `InlineContext`, `Hybrid`, `LocalToolRefs`, or `NoSourceVault`. This makes it possible not to treat a cloud-backed model with a local tool harness, such as Claude Code, the same as a local model in LM Studio, separating provider trust from data release.

In particular, even though Claude Code/Codex can operate the local filesystem and shell, model inference itself is cloud/remote, so MCP release decisions must not automatically treat them as `LocalLLM` and return high-privacy bodies. Conversely, trusted local/private models can be given a higher access ceiling according to their registered `AccessProfile`.

`SourceVault_mcp.wl` is not the MCP protocol transport itself; it is the Wolfram Language-side library responsible for tool schemas, `initialize/tools/list/tools/call` dispatch, argument validation, provenance, and JSON-safe responses. The HTTP/JSON-RPC transport is handled by a Python proxy, which passes requests via a file command queue to `SourceVaultMCPDispatch` in the service kernel. MCP is therefore not the authority over the SourceVault data store but a safe projection layer. Final decisions on read/write/cloud-send are delegated to the release policy, `AccessGrant`, `NBAccess`, and so on.

図8 MCPを介したSourceVaultのURI参照・Directive共有・Artifact交換

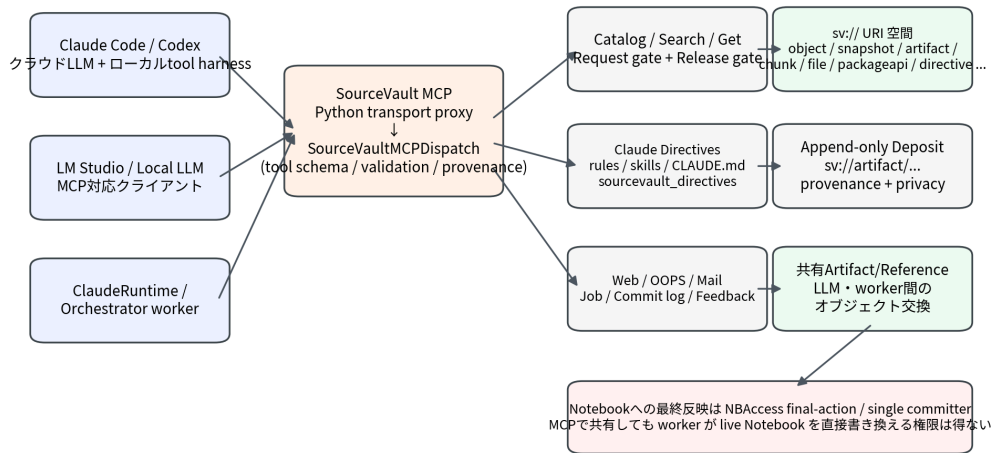


Figure 7. URI references, Directive sharing, and Artifact exchange via MCP

The current SourceVault MCP tool set goes beyond a mere search API. Below are the 26 tools organized by function.

Category	tool	Role
Universal core	sourcevault_catalog / sourcevault_search / sourcevault_get	adapter catalog, cross-adapter search, single-URI retrieval
Access	sourcevault_request_access / sourcevault_access_status	requesting and polling the AccessGrant needed for bodies etc.
Capability	sourcevault_runtime_capabilities	returns the per-model/client PromptDeliveryProfile and available capabilities
Web	sourcevault_web_search / sourcevault_submit_web_search / sourcevault_job_status / sourcevault_job_result / sourcevault_get_document	synchronous/asynchronous web search, jobs, fetched documents
Filesystem	sourcevault_fs_list / sourcevault_fs_read	listing/reading of allow-listed package/working directories. Secret-looking files are refused
Directives	sourcevault_directives	lists the rules/skills/root instructions of Claude Directives and returns them as URIs
Package history	sourcevault_commit_log	package commit metadata from the GitHub API
OOPS	sourcevault_oops_status / sourcevault_oops_search_threads / sourcevault_oops_thread	OOPS corpus status, thread search, thread retrieval
General mail	sourcevault_mail_status / sourcevault_mail_search_threads / sourcevault_mail_thread	search over structured general-mail data
Workflow authoring	sourcevault_workflow_write	helper for API/LM Studio-type clients to safely write workflow files under SourceVault_workflows
Orchestrator feedback	sourcevault_orchestrator_help / sourcevault_orchestrator_subtask	bridges help requests and subtask proposals to the Orchestrator side
Feedback	sourcevault_feedback_submit	append-only feedback envelopes for critiques, corrections, help, etc.
Write	sourcevault_deposit	append-only deposit of LLM-generated artifacts

sourcevault_search spans adapters, normalizes the SearchSpec, and passes each result through the request-time release gate.

`sourcevault_get` resolves an `sv://` URI and by default returns only low-leakage summary/metadata. As a rule, `body/raw/context` require an `AccessGrant`, and sending confidential bodies to a cloud sink is refused even with a grant. That is, an MCP client's ability to search is separated from its ability to retrieve bodies. As an exception, however, `view="body"` for `PublicDoc` documents whose adapter declares `BodyGrantRequired -> False` requires no grant and passes only through the release gate. The built-in `packageapi` adapter uses this exception and can return the bodies of `PrivacyLevel 0` package API documents even to cloud clients. This exception is limited to `body` and does not make general `raw/context` grant-free.

5.9 MCP Sharing of Directives and Artifact Deposit

5.9.1 The Significance of Sharing Claude Directives as an MCP Database

Claude Directives on the `claudecode` side is the canonical directive repository consisting of `CLAUDE.md`, rules, and skills; it can be generated and synchronized (materialized) as real files into `.claude/` for the Claude CLI, and into `AGENTS.md`, `.agents/skills`, etc. for Codex. Claude Code and Codex can use this local harness directly. Meanwhile, `sourcevault_directives` in `SourceVault` MCP exposes the same directive set as searchable, referenceable data, returning rules/skills as references such as `sv://file`.

This allows local LLM clients such as LM Studio, which can call MCP tools during inference, to reference the same rules and skills as Claude Code/Codex when needed. The significance is that directives are projected from `SourceVault/MCP` as a single managed object, rather than manually duplicating separate system prompts per model. However, which directives are disclosed to which model is subject to `AccessProfile` and release policy; MCP visibility alone does not unlock all data.

5.9.2 Writing via MCP — `sourcevault_deposit`

`sourcevault_deposit` is an append-only API by which an LLM actively saves generated markdown, Wolfram Language, PDFs, images, JSON, etc. under `SourceVault` management. It is not a mutation that directly permits overwriting or deleting existing objects or relaxing their privacy. At commit time it requires an `AccessGrant` with the `DepositArtifact` permission or a model `AccessProfile`, and it records provenance (`authoredBy`, `inputRefs`, `citationRefs`, `promptRefs`) plus the spec-level `SessionId` and `BatchId`. On success an `sv://artifact/...` is returned, and the same idempotency key with the same content is treated as a replay.

Privacy is not based solely on the LLM's own declaration; it inherits the maximum privacy of the inputs actually released in that `Session/Batch`. When `sourceRefs` exist but there is no observed evidence, the fail-closed floor (privacy 0.75) is applied. The design is thus oriented toward preventing privacy laundering, in which an agent that has read private sources self-declares its output as "public."

5.9.3 Using `SourceVault` as an Object-Sharing Surface Between LLMs and Workers

Under `ClaudeOrchestrator`'s principles, workers cannot commit directly to live Notebooks; deliverables cross the boundary as artifacts. Meanwhile, MCP deposit lets an LLM save an artifact into `SourceVault` and issue an opaque reference of the form `sv://artifact/...`, and another LLM/worker can retrieve the permitted projection through MCP `search/get`. Combined, these allow `SourceVault` to be used as a provenance-carrying object-exchange surface in place of "directly editing Notebooks as shared memory."

This does not mean that, at present, "all inter-worker communication has already been automatically unified onto MCP/SV URIs." Rather, it is a structuring principle for inter-agent sharing without breaking the single-committer constraint. It is also consistent with the `Orchestrator`'s session/episode design, which passes large artifact bodies by reference instead of embedding them in Petri tokens or conversations. Final reflection into the Notebook still goes through the `reducer/committer` or `NBAccess`'s `PendingFinalActionQueue`.

Primary code and documentation references: `SourceVault_info/docs/user_manual.md` / `SourceVault_info/docs/api_mcp.md` / `SourceVault_mcp.wl` / `SourceVault_info/design/sourcevault_universal_mcp_access_spec_v2.md` / `claudecode_info/docs/api_directives.md` / `ClaudeOrchestrator_info/design/claude_orchestrator_runtime_session_episode_petri_spec_v0_1.md`

5.10 Assessment

Even at the stage of its foundational architecture, SourceVault is not a mere document repository. It integrates immutable snapshots, claims, Evidence Bundles, Notebook management, PromptRouter, and release contexts, providing source-centric evidence trails and deterministic re-execution. It is because of this foundation that the mining layer described in Chapter 8 can preserve not merely "finding similar text" but which snapshot, span, person, topic, and workflow a relation derives from.

5.11 Future Directions

For the SourceVault core going forward, it is important to extend the `sv://` URI resolver to all object classes and to stabilize alias resolution against legacy refs. If URIs become the common key for search, MCP, workflows, artifact relations, and citations, differences in internal IDs need not be exposed to external clients.

There is also room for more automatic evaluation of source lifecycle and staleness propagation of derived artifacts. If, when a snapshot is updated, the system could compute from the dependency graph which claims, summaries, workflow results, and Notebook descriptions are subject to re-verification and push them to a review queue, its value as a long-term knowledge infrastructure would increase.

The more MCP deposits and agent artifacts grow, the more important it also becomes not to confuse self-generated outputs with external primary sources. `ArtifactType`, `SourceRefs`, `AuthoredBy`, `DepositedBy`, confidence, and review state must be clearly separated, and the design of not automatically promoting LLM-generated outputs to evidence must be maintained.

Chapter 6. Code-Level Integration Analysis

This chapter reads out, at the code level, the design principles common to the four repositories. Viewed individually, each package appears to be an independent feature, but there is a consistent philosophy in how permissions are composed, how proposal and execution are separated, how state is held, and how execution locations are chosen. The significance of making these common principles explicit is that they provide criteria for deciding "what should be implemented where" as packages continue to grow.

The most important principle is the idea that "permissions are not added from upper layers to lower ones; the base permissions of the lower layer are narrowed by upper scopes." Because Runtime scope, Workflow scope, and Step scope are layered onto NBAccess's base policy as an intersection, convenience features in upper layers cannot easily extend permissions beyond the safety boundary. The second principle is to separate proposal from execution, artifacts from live state, and immutable records from current projections. LLMs return proposals, workers return artifacts, SourceVault keeps append-only events, and the current state is projected from them. These may look like separate features, but they share the common design philosophy of "never making intermediate results directly authoritative." The third principle is treating the execution location as part of policy as well. Main kernel, subkernels, wolframscript, SpecificMachine, and remote PCs are chosen not only for performance but according to FrontEnd dependence, secret access, license seats, and fault isolation.

In what follows, Sections 6.1–6.5 show the implemented forms of each principle, Section 6.6 lists the responsibility boundaries, and Section 6.7 summarizes future directions.

6.1 Design That Concentrates Safety at the Lower Boundaries

Looking at the four repositories at the code level, a consistent structure emerges in which upper layers delegate decisions to the lower boundaries rather than re-implementing safety on their own.

```
SourceVault / Orchestrator / Runtime Scope
      ↓ intersect
    NBAccess Base Policy
      ↓
    Final Decision
```

The final permission is conceptually the following intersection.

```
FinalPermission
= NBAccessBase
n RuntimeScope
n WorkflowScope
n StepScope
```

However, this is a [Conceptual model]; no single common policy-composition function of this name is implemented. In practice, each layer implements the same principle in a different vocabulary: NBAccess with PrivacySpec/AccessSpec, the Orchestrator with the Capabilities of TaskSpec, and SourceVault with ReleaseContext and CapabilityLease. What they share is the directionality that Runtime and Orchestrator can narrow permissions but cannot widen them beyond NBAccess's base permissions; unifying these into a single policy digest/contract remains future work (see 6.7).

6.2 Separation of Proposal and Execution

LLM output is a proposal, not a command.

```
LLM
↓
Proposal
↓
Parse
↓
Hold
↓
Validate
↓
Decision
↓
Execute
```

This principle is common not only to the Runtime but also to PromptRouter, the Orchestrator planner, and multi-model adjudication.

6.3 Separation of Artifacts and Live State

Workers do not modify live Notebooks; they return artifacts.

```
Worker Result
= Payload
+ Sources
+ TaskRef
+ Schema
+ Validation State
```

Commit to live state happens in a separate stage. This structure is also consistent with SourceVault's derived artifact model.

6.4 Immutable and Append-Only

SourceVault's raw snapshots, claim events, Knowledge Home appends, TagAssertions, AuthorshipAssertions, workflow events, and the like are managed as immutable or append-only wherever possible.

When correcting, old records are not deleted; instead, Superseded, Retracted, decision events, and the like are appended.

```
Original Record
↓
Superseding Event
↓
Current Projection
```

Each projection implemented as event-sourced can be regenerated from the event sequence. However, this property does not unconditionally apply to the entire current state of SourceVault. Registry persistence files (e.g., search profiles are saved to and restored from `PrivateVault/config/searchindex-profiles.wxf`), machine-local config, access-status sidecars, and caches/pointers have persistence contracts separate from the event log.

6.5 Task Placement and Resource Places

Type	Main conditions
Main Kernel	FrontEnd, Notebook, UI, or session state is required
Subkernel	pure computation, parallelizable, no Notebook side effects
WolframScript	long-running, batch, heavy I/O, an independent process is preferable

The Runtime makes recommendations, NBAccess validates actions, and the Orchestrator permits execution as resource places on the workflow.

6.6 Responsibility Boundaries Across the Four Repositories

Concern	Canonical owner
Confidentiality of Notebook cells	NBAccess
Whether a held expression may be executed	NBAccess
Progression of a single turn	ClaudeRuntime
Task placement recommendation	ClaudeRuntime
Dependencies among multiple tasks	ClaudeOrchestrator
Worker permissions	ClaudeOrchestrator + NBAccess
Notebook commit	ClaudeOrchestrator single committer
Sources and snapshots	SourceVault
Claims and evidence	SourceVault
Search release	SourceVault
identity	SourceVault_identity
Long-term topic graph	SourceVault mining/Knowledge Home

These boundaries are the premise for understanding the extensions in the latter half.

6.7 Future Directions

Going forward, it is desirable to promote these common principles from coding conventions into machine-verifiable contracts. If invariants such as "upper layers do not extend NBAccess permissions," "workers do not hold NotebookWrite," and "source-derived artifacts require a provenance ref" could be run as contract tests, it would be easier to prevent design erosion as packages multiply.

Another task is to thread a common run/step-level ID through all layers, binding PromptRun, Runtime events, Workflow runs, MCP calls, CapabilityLeases, and SourceVault artifacts into a single trace. This would enable not only failure analysis but also a consistent explanation of "which LLM input, which retrieval span, and which workflow worker this Notebook output derives from."

As multi-PC operation advances, principles that were effective locally — single-writer, atomic claims, and idempotency keys — must also be maintained on the shared vault. Since synchronized storage such as Dropbox is not a distributed DB, the current policy of basing things on per-machine logs and merge views needs to be further systematized.

Chapter 7. Evolution from the Four-Layer Stack into a Knowledge-and-Execution Operating Model

The four layers described in the preceding chapters are, on their own, already complete as a system that securely connects confidential Notebooks to LLMs, advances complex work across multiple workers, and stores referenced sources and generated products in a reproducible form. However, once URIs, MCP, mining, identity, AutoTrigger, diagnostics, and cognition are added to SourceVault, the individual functions begin to circulate: retrieved knowledge launches workflows, workflow results return to SourceVault as artifacts, and those artifacts are referenced by the next LLM or agent. The purpose of this chapter is to present this circular structure and the mechanisms for autonomous execution and multi-PC coordination that support it, while distinguishing the parts that are already implemented from those still at the design stage.

The key to making this loop work is using `sv://` URIs and SourceVault as a shared surface. LLMs and workers do not write directly to Notebooks; they deposit artifacts into SourceVault, and other workers retrieve them via URIs. Because the final reflection into the Notebook is performed by the single committer, control over live state is not lost even as autonomy increases. AutoTrigger and SystemDoctor transform this loop from a system that runs only when a person presses a button into one that runs autonomously in response to state changes and schedules. What matters, however, is the structure that places a health gate before autonomous execution and halts general workflows when diagnostics fail.

Importantly, this evolution does not replace the existing four layers; it is possible precisely because of the separation of responsibilities confirmed in the first half of this report. Even if the mining layer generates large numbers of relations between materials, the release gate decides whether they may be viewed or sent. Even if an LLM proposes expressions using neighboring materials, NBAccess decides whether they may be executed. Even if multiple workers generate deliverables, the single committer performs the reflection into the Notebook.

In what follows, Section 7.1 presents the overall picture of the closed loop, Sections 7.2–7.3 describe autonomous triggering and the health gate, Sections 7.4–7.5 describe the current state of multi-PC coordination, and Section 7.6 summarizes future directions.

7.1 Prompt, Workflow, MCP, and SourceVault Form a Closed Loop

Connecting the mechanisms of the preceding chapters extends interactive LLM assistance into the following closed loop. A user or an external LLM retrieves evidence from SourceVault via `sv://` URIs, processes it through PromptRouter or a workflow, and returns the deliverable to SourceVault as an artifact. Another worker references that artifact, and after verification the single committer reflects it into the Notebook. Furthermore, if the processing itself is registered as a TriggerSpec, the next run can be automated in response to a time of day or a SourceVault event.

```
SourceVault source / sv:// refs
  ↓ search, MCP, PromptRouter
Runtime / Orchestrator workflow
  ↓
artifact deposit / snapshot / evidence
  ↓
AutoTrigger monitors schedules and events
  ↓
next workflow run
  ↓
results back into SourceVault
```

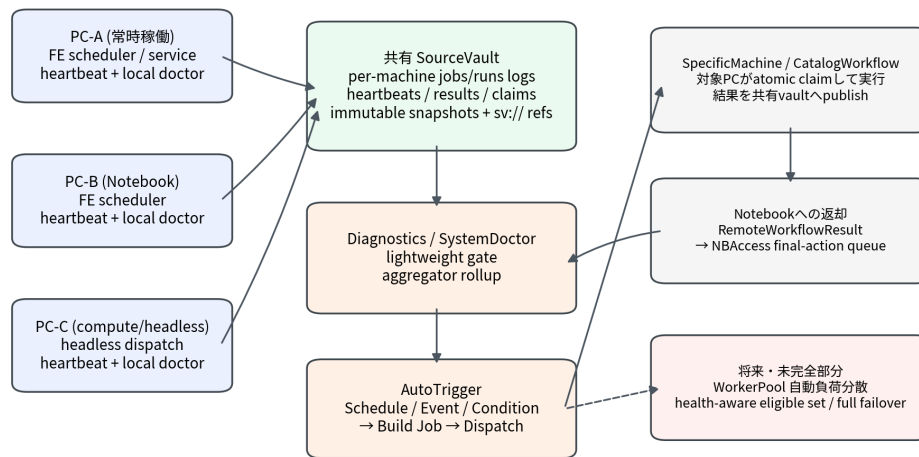
Once this loop is in place, SourceVault changes in character from a static vault into a shared working surface that LLM agents read and write and that workflows continuously update. Autonomy, however, does not mean unlimited authority: it is realized inside boundaries where reads pass through the release gate, writes are append-only deposits or approved mutations, the Notebook is handled by the single committer, and external actions are governed by capability/approval.

7.2 SourceVault_autotrigger — Launching Workflows Automatically and Safely

Rather than inflating cron into a monolith, SourceVault_autotrigger separates the Trigger Registry, the Scheduler/Watcher, an append-only Job Queue, and the Executor. A TriggerSpec has Schedule, Condition, Target, ExecutionPlacement, DiagnosticsPolicy, and RunPolicy; merely being saved does not cause automatic execution, and conditions such as Enabled and AutoRunCapability must be satisfied.

As of the current phase 1.4, implementation has progressed through trigger validation and registration, schedule/condition matching, the diagnostics gate, job construction on ticks, dispatch of subkernel/Workflow/CatalogWorkflow, scheduler registration on the shared polling tick, SpecificMachine placement, and remote result collection. PromptRoute is main-kernel gated and, as a rule, is not executed automatically from automatic ticks. WorkflowRoute/WorkflowTemplate and CatalogWorkflow can be launched asynchronously when their conditions are met.

図9 AutoTrigger・SystemDoctor・共有SourceVaultによる複数PC協調



実装済みのSpecificMachine実行・per-machine single-writer log・atomic claimと、仕様上のWorkerPool/failoverを区別して示す。

Figure 8. Multi-PC Coordination via AutoTrigger, SystemDoctor, and the Shared SourceVault

7.3 Designing SystemDoctor as the First Automated Workflow

What matters in the AutoTrigger specification is the policy of automatically running a comprehensive SystemDoctor workflow before any general business workflow. Specification v0.1-r9 positions Diagnostics/SIEM as the prerequisite layer for automatic execution and defines that "the first workflow to be auto-triggered is not a user workflow but a system doctor workflow using SourceVaultSystemDoctor/SourceVault_diagnostics." Enabled=True for a general workflow presupposes that the most recent health of the required components is OK or an explicitly tolerated Degraded.

A distinction from the current implementation is necessary, however. SourceVault_diagnostics implements the cross-package collector/store/doctor, license probe, service health, machine heartbeat, aggregator rollup, and so on, and AutoTrigger uses SourceVaultDiagnosticsLightweightDoctor as a component-scoped gate before dispatch. Meanwhile, api_diagnostics.md still lists the automatic comprehensive-doctor trigger as a deferred item. We therefore assess that "SystemDoctor first" is the canonical design policy, and that at present the lightweight doctor gate has been implemented ahead of it.

7.4 Sharing SourceVault Across Multiple PCs

When a shared SourceVault is operated across multiple PCs, SourceVault itself becomes a durable common recording surface. For Diagnostics, each PC writes heartbeats to its own dedicated path, and the Aggregator rolls up the health of all machines. The AutoTrigger jobs/runs logs have likewise been changed from a scheme in which all PCs append concurrently to a single shared JSONL to per-machine single-writer files, `autotrigger/jobs/<machineTag>.jsonl` and `autotrigger/runs/<machineTag>.jsonl`. This is a countermeasure to an observed problem in which conflicted copies arose on Dropbox and similar services and remote appends were lost.

For a `SpecificMachine CatalogWorkflow`, the requesting PC submits "run this slug on this machineTag" to the shared job log, and the target PC's FE scheduler or headless dispatch picks it up. Before execution, an atomic claim (atomic creation of `autotrigger/claims/<slot>/`) prevents duplicate execution; the license process slot is checked; and upon completion, `output.wxf` is published to the shared vault. Notification back to the requesting Notebook goes through `NBAccess's PendingFinalActionQueue`, so even remote execution does not break the single-committer principle.

Feature	Current State	Implications for Redundancy and Load Balancing
per-machine heartbeat	Implemented	Observes each PC's liveness independently
aggregator rollout	Implemented	Global health and active-aggregator candidates
per-machine jobs/runs	Implemented	Avoids concurrent-append conflicts on shared storage
<code>SpecificMachine</code> dispatch	Implemented	Places jobs on specific PCs such as GPU machines or always-on machines
atomic dispatch claim	Implemented	Prevents duplicate execution by the FE scheduler and the headless dispatcher
remote result publication	Implemented	Retrieves results executed on another PC via the shared vault
license seat guard	Implemented	Defers when free process slots are insufficient and retries automatically
Automatic WorkerPool load balancing	Not implemented / under design	A future layer that automatically selects among multiple candidate PCs based on load and health
full failover	Partially designed	Degraded operation and handover across multiple always-on machines remain to be integrated

7.5 What Redundancy and Load Balancing Have Achieved So Far

Combining the above, the path of requesting a workflow on PC A, executing it on PC B, publishing the result to SourceVault, and retrieving it from PC A's Notebook is already implemented. Headless compute nodes can also participate via opt-in dispatch. This is not static file synchronization but the foundation of distributed execution encompassing diagnostics, placement, claims, and result publication.

On the other hand, the capability to automatically place `EnvironmentIndependent/WorkerPool` jobs on the optimal PC based on health, load, available models/GPUs, and license capacity, and to fully fail over to another node when the active node goes down, is not yet complete. The `AutoTrigger` API documentation itself explicitly states that WorkerPool load balancing and failover are unimplemented. This report therefore assesses that the foundation for achieving multi-PC redundancy and load balancing is largely implemented and `SpecificMachine` distribution is at a practical stage, while the general-purpose WorkerPool scheduler remains a future feature.

Primary code and documentation references: `SourceVault_info/design/sourcevault_auto_trigger_scheduler_spec_v0_1.md` / `SourceVault_autotrigger.wl` / `SourceVault_info/docs/api_autotrigger.md` / `SourceVault_info/docs/api_diagnostics.md` / `SourceVault_info/docs/user_manual.md`

7.6 Future Directions

The direction of future development is the completion of an event-driven knowledge execution fabric centered on SourceVault. By launching workflows conditioned on SourceVault events, schedules, approval states, and queue states, and saving the results again as `sv://` artifacts, points of human intervention can be limited to the necessary approval boundaries alone.

Positioning the comprehensive `SystemDoctor` as the first automatically triggered system workflow, and deciding whether other workflows may run based on its results, is an operationally clear configuration. At present, however, the lightweight doctor gate has come first, and the automatic trigger for the comprehensive doctor and the fan-out of all producer probes are still under development, so progress must be made while distinguishing what is implemented from the future vision.

For multiple PCs, introducing WorkerPool, health-aware eligible sets, load balancing, and failover in addition to SpecificMachine execution would allow the shared SourceVault to evolve into a small-scale distributed execution platform. Given the characteristics of shared file synchronization, however, distributed locks must not be over-trusted; per-machine single-writer and atomic claim must be made canonical.

Chapter 8. The SourceVault Mining Layer

The SourceVault mining layer is the part that, rather than merely full-text searching the accumulated materials, discovers relations among them and transforms them into a knowledge space that can be explored as neighborhoods. Whereas ordinary vector search centers on placing similar documents close together, SourceVault combines explainable relations such as topic, author, sender, quote, reply, explicit link, derivation, and retrieval history. The aim of this design is to create an exploration space that can present, with supporting evidence, why a given material is a candidate — a character distinct from opaque embedding neighborhoods.

What matters as the initial coordinates of this knowledge space is the OOPS corpus. Because it contains roughly 6,500 mails and roughly 4,100 topic items, along with hand-annotated mail-topic correspondences, topic relations, and quote relations, it can be used not merely to search past mail but as a seed for positioning new general mail, Notebooks, PDFs, and Web pages within the existing topic space. From the user's perspective, this appears as Neighbor Hopping: the material currently being read is converted into a query, and next candidates are presented from three neighborhoods — shared topics, direct quote/reply, and relation-expanded topics. Even without recalling search terms, one can dig through knowledge while moving from material to material.

In what follows, Sections 8.1–8.2 describe the definition of mining and the multi-stage structure of search, Section 8.3 the mechanism of neighborhood search, Section 8.4 cross-medium projection and identity, Section 8.5 the structure of the OOPS corpus, and Sections 8.6–8.7 the extension to general mail and privacy boundaries; Section 8.8 summarizes the implementation status and Section 8.9 future directions.

8.1 Definition of Mining

Mining in SourceVault is not the process of merely adding sources to the set of searchable targets. It is the process of discovering, proposing, and confirming relations among objects — mail, local files, Web, arXiv, Eagle, Notebooks, PDFs, generated artifacts — such as topic, tag, author/sender/creator, person/organization, quoting/quoted-by, reply/session, explicit link, derivation source/generation target, workflow input/output, past search-usage history, safety and privacy, and error and review states.

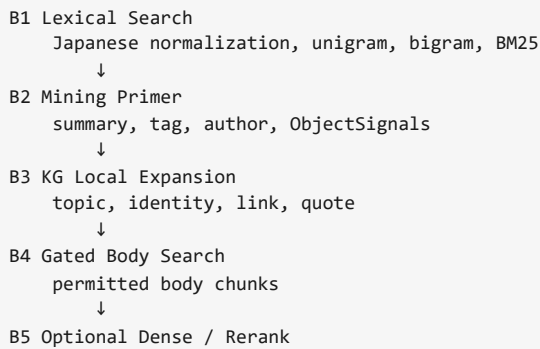
Mining results are stored as MiningObjects, carrying Confidence, TargetRefs, InputRefs, GeneratedByRunID, ReviewState, and so on. What matters is distinguishing mining results from the original materials.

```
Original Object
  ↓
Mining Assertion / Proposal
  ↓
Human Review or Deterministic Validation
  ↓
Search Projection
```

Even when an LLM infers that two identifiers appear to be the same person or that two items appear to share the same topic, that inference is not immediately turned into canonical fact.

8.2 The Multi-Stage Structure of Search

Search in SourceVault conceptually has the following stages.



Summaries and mining results are used not as the evidence for the final answer but as primers for narrowing candidates. The final answer must include original-text chunks or body grounding.

8.3 Neighborhood Search and Neighbor Hopping

8.3.1 TopicPosition and Evidence-Backed Weights

In ordinary search, the user enters search terms. In neighborhood search, the object currently being viewed itself becomes the starting point of the next search. First, the target object is placed in the topic space. Suppose, for example, that the TopicPosition of a certain PDF is as follows.

Current PDF		Another PDF	
cellular automata	0.72	reversible computing	0.65
reversible computing	0.48	cellular automata	0.59
information thermodynamics	0.31	Fredkin	0.25
workshop	0.18	logic circuits	0.21

Because the two share cellular automata and reversible computing, they become neighborhood candidates.

TopicPosition, however, is not a mere opaque embedding. Each topic weight carries supporting evidence (a Method).

Method	Meaning
OwnerConfirmed	Explicitly confirmed by the owner
ExplicitOOPS	Explicit marker in the OOPS body text
SeedMatched	Surface-form match against the seed dictionary
RelationExpanded	Expanded from the relation graph
AutoExtracted	New topic candidate
LLMProposed	Proposed by an LLM, with a low review state

The fraction of paragraphs that cannot be explained by existing topics is retained as UnknownMass, so that not everything is force-classified into existing categories.

8.3.2 Three Kinds of Neighborhoods

Neighborhoods are organized into the following three kinds.

The first is the topical neighborhood: objects that share the same topic item. Whether the medium is mail, PDF, Notebook, or Web, they can be connected as long as their topic projections are common.

The second is the structural neighborhood: objects with direct relations such as Reply, Quote, QuotedBy, Attachment, Link, Citation, GeneratedFrom, Supersedes, and WorkflowInput.

The third is the relation-expanded neighborhood: starting from central topics, the topic relation graph is traversed to find objects that carry related topics.

```

cellular automata
├ reversible computing
├ self-reproduction
├ artificial life
└ Reservoir Computing

```

Expansion is not unbounded; it is limited by hop count, node count, edge weight, and per-node top-k. OOPS auto-tagging mainly implements weighted 1-hop expansion, while KG expansion for search implements cycle-safe multi-hop BFS.

8.3.3 Neighbor Hopping and Three-Ring Proposals

SourceVaultKnowledgeHomeNeighborhoodSearch computes the TopicPosition of the target object and returns mails and Knowledge Home paragraphs whose topics overlap.

```

Current mail
↓
Mail sharing the same topic
↓
Another mail quoted by that mail
↓
Knowledge Home appends on related topics

```

Neighbor Hopping is the operation of moving from the current material to a neighbor and then exploring with that material as a new starting point.

In Knowledge Home, candidates for what to read next are divided into three rings.

```

Ring 3: relation-expanded neighborhood
Ring 2: direct quote / reply / link
Ring 1: shared central topics
Current Object

```

Ring 1 consists of materials sharing the central topics; Ring 2 is the direct neighborhood of quoting, quoted-by, reply, and explicit links; Ring 3 consists of materials carrying topics reached by expanding the relation graph from the central topics. Each candidate is annotated with Reasons, so one can check why it was proposed.

8.4 Cross-Medium Common Projection and Identity

8.4.1 Common Projection

SourceVault does not erase differences between media; it creates a common projection while preserving each original.

```

ObjectURI
ObjectClass
Title
Body / Chunk
Summary
Tags
TopicRefs
Authors
Links
PrivacyLevel
SafetyState
SourceEpoch
DerivedFrom

```

Medium	Primary Units	Structural Relations	Identity Observations
Mail	Mail, Paragraph, Quote, Session	Reply, Quote, Attachment	From, To, Cc
Notebook	Notebook, Section, Cell, Output	Order, evaluation, generation	Creator
PDF	Document, Page, Section, Figure	Citation, Version	Author, XMP Creator

Medium	Primary Units	Structural Relations	Identity Observations
Web	Page, DOM Block, Article	URL, Link, Snapshot	schema.org Author
arXiv	Paper, Version, Reference	DOI, Citation	arXiv Author, ORCID
Artifact	Output, Run Result	GeneratedFrom	LLM, Agent, Workflow

This makes it possible, for example, starting from a Notebook cell, to build a structure that traces to PDFs on the same topic, Web pages by the same author, and the mail that served as the quotation source.

8.4.2 SourceVault_identity — Identifiers and Entities

In person identification, the same person has multiple representations.

```
imai@example.jp
Katsunobu Imai
今井克暢
K. Imai
GitHub: transreal
Notebook Creator: imai_
```

SourceVault_identity separates these into two layers: Identifier and Entity. An Identifier is raw observed identification information such as Email, URI, SNS, PersonName, OrganizationName, ORCID, arXivAuthor, DOIName, WebAuthorProfile, DocumentCreator, and LLMOrAgent. An Entity is a real-world subject — Person, Organization, Bot, Service, MailingList, and so on — that bundles multiple Identifiers.

The From, To, and Cc of a mail are registered as Identifiers at ingest time. Because a deterministic ID is generated from the same kind and normalized value, repeatedly ingesting the same email address upserts to the same Identifier.

8.4.3 AuthorshipAssertion and EntityLinkProposal

Relations between media and persons are projected onto common assertions.

```
Mail      — Sender — Identifier / Entity
PDF       — Author — Identifier / Entity
Notebook  — Creator — Identifier / Entity
Web       — Author — Identifier / Entity
Artifact  — Agent  — Identifier / Entity
```

When it has not been established that two identifiers belong to the same person, an EntityRef is not set directly on the Identifier; instead, the link is stored as an EntityLinkProposal. Proposals carry a score, features, provenance, and a review state, and a human accept/reject is not silently overturned by later automatic rescoring.

Identity resolution is not a security boundary. The fact that someone appears to be a known person is not a reason to lower a privacy level or permit cloud release.

8.5 The OOPS Corpus — Initial Reference Coordinates of the Knowledge Space

8.5.1 The Corpus and Seed Indexes

OOPS is a corpus of roughly 6,500 mails from 1992 to 2005 and roughly 4,100 topic items. SourceVault uses it not as mere additional mail data but as the initial reference coordinates of Knowledge Home.

The main data are as follows.

Data	Role
item-name.index	Topic item names and aliases
mail-to-item.index	Mapping from mails to topics
item-to-mail.index	Mapping from topics to mails (specified in the search foundation spec; parser implementation is forthcoming)

Data	Role
item-relation.index	Weighted directed relations between topics
item-relation-up.index	Reverse-direction topic relations
quote-table.index	Quoted source mails
quoted-table.index	Quoted-by relations
mail-info.index	Mail number, list, author
oops*.txt	Mail body text

mail-to-item.index is hand-annotated gold data and can serve as a reference standard for topic estimation and search evaluation.

8.5.2 Owner-scoped topic namespace

OOPS topics are not global keywords.

```
[ki 2573]
[mi 2573]
[aga 2573]
```

are topics in separate namespaces. In the initial configuration, ki is mapped to the current owner, while mi, aga, and others are retained as namespaces of other owners. Even identical labels are not automatically merged when their owners differ.

```
TopicItemRef
Namespace
LocalId
OwnerRef
NamespaceKind
CanonicalLabel
SurfaceForms
PrivacyLevel
SourceRefs
ReviewState
```

Unknown namespaces are not discarded either; they are retained as Missing["UnknownOwner"].

8.5.3 Internal Mail Structure and Topic Assignment

In the OOPS model, a mail is not treated as one long string.

```
Mail
├─ Header
├─ Prose Paragraph
│   └─ Primary Topic
│       └─ Secondary Topic
├─ Quote Paragraph
│   └─ QuotedFrom
├─ Signature
└─ Footer
```

The priority order for topic assignment is ExplicitOOPS, SeedMatched, RelationExpanded, AutoExtracted, LLMProposed.

Explicit markers such as ☺, ○, •, and [namespace id] are treated as hand-annotated information of the highest quality. When the seed dictionary matches the body text, the assignment becomes SeedMatched, and the relational neighborhood of that topic can be added as RelationExpanded with lower weights. Terms absent from the seeds become AutoExtracted candidates and are not promoted to official topics until owner confirmation.

8.5.4 The Quote Graph and Mail Sessions

OOPS mails contain markers such as the following.


```
-*- Quote (from 6539) -*-  
...  
-*- Unquote -*-
```

The direction of a quote edge is from the quoting mail to the quoted mail. The seed relations in quote-table.index are treated as the authoritative graph; fuzzy matching intended for general mail does not overwrite existing seed relations.

From quote edges, subject lines, and reply relations, multiple mails are grouped into sessions.

```
Mail Session  
├ Initial Proposal  
├ Reply  
├ Counterproposal  
├ Decision  
└ Later Reference
```

Creating search chunks at the session level rather than per single mail makes it possible to look for things such as the final schedule, the conclusion of a discussion, or unresolved items. Session summaries alone, however, are not used as the final evidence; the relevant paragraphs and quote edges are retained as grounding refs.

8.6 Extension to General Mail and Topic Growth

The purpose of OOPS is not to convert all general mail into OOPS's old file format. The purpose is to project general mail onto the same topic/quote/session model.

```
General Mail  
├ Paragraph segmentation  
├ Quote / Forwarded block extraction  
├ Reply / Reference relations  
├ Matching against OOPS seeds  
├ New topic candidates  
├ Mail Relation Graph  
└ Topic Graph
```

Structuring of general mail is seed-optional. If OOPS seeds exist, existing topics are reused; if not, a TopicVocabulary is grown from the mail corpus. To avoid a circular dependency between vocabulary and sessions, processing proceeds in two passes.

```
Pass A  
Mail-level Vocabulary  
    ↓  
Relation Graph and Sessions  
    ↓  
Pass B  
Session-aware Vocabulary Refinement  
    ↓  
Paragraph Topics and Topic Graph
```

Topic growth is handled within the same framework. Because OOPS is a vocabulary that extends only up to 2005, terms such as ChatGPT, Claude Code, ComfyUI, Agentic Search, DGX Spark, and Meta Quest 3 may not exist in the original seeds. These are extracted as AutoExtracted candidates and, after owner confirmation, promoted to new topic items.

```
Unknown Surface  
    ↓  
AutoExtracted Candidate  
    ↓  
Owner Confirmation  
    ↓  
New Topic Item  
    ↓  
Re-index  
    ↓  
Future SeedMatched
```

OOPS is therefore not a fixed classification table but an initial map that grows through current mail, Notebooks, PDFs, and the Web.

8.7 Privacy Boundaries

Being in the neighborhood does not mean being viewable.

Near

≠ Readable

≠ Cloud-sendable

≠ Publicly exportable

≠ Safe to execute

OOPS ingest uses not only the list name but also the actual To, Cc, Delivered-To, and other headers for privacy determination. Materials on private lists or containing third-party information are tagged with PrivateML, ThirdPartyContent, NoCloudLLM, NoPublicExport, and so on.

If even one high-privacy mail exists within a session, the PrivacyLevel of the entire session is the maximum and its tags are the union. Score boosts from mining or identity change only the ranking and do not loosen the release gate.

8.8 Implementation Status

According to the handoff document of July 14, 2026, the following are implemented and verified for Knowledge Home.

Phase	Content	Status
1A	Read-only browser for topics, mails, and quotes	Implemented and verified
1B	Non-destructive appends, ULID, alias, supersede, undo, merge	Implemented and verified
1C	TopicPosition, UnknownMass, neighborhoods, three-ring proposals	Implemented and verified

Meanwhile, the stage of fully spanning PDFs, Notebooks, the Web, and general mail with a single NeighborhoodSearch is in the midst of connecting the common model to the per-medium projections.

8.9 Future Directions

The biggest challenge ahead is making the neighborhood search implemented for OOPS mail and Knowledge Home paragraphs span fully across Notebooks, PDFs, the Web, general mail, and MCP artifacts. Each medium's native structure must be preserved while being projected onto common TopicPositions, AuthorshipAssertions, and Relation edges.

The second challenge is quality evaluation of mining results. Since AutoExtracted topics, EntityLinkProposals, and relation expansion can produce false positives, a mechanism for self-correction using human review, confidence, and rejected history is needed. With OOPS's hand-annotated data as ground truth, not only recall and NDCG (Normalized Discounted Cumulative Gain, a metric of ranking quality) but also the false-link rate can be evaluated.

The third challenge is the connection to agentic search. When MCP clients or LLM agents perform Neighbor Hopping, they must not rely on primers and summaries alone as final evidence but must ultimately ground on gated bodies/spans. This maintains evidence quality and privacy boundaries even as the freedom of exploration increases.

Chapter 9. Defense in Depth Across Safety, Diagnostics, and Cognitive Support

This chapter reads safety issues, including Prompt Injection, not as a single detector but as defense in depth. SourceVault does not assume that attack text can be detected 100% of the time. The aim of the design is not to "see through malicious input" but to build a structure in which damage does not spread even if something is missed. That is, the authority of inputs is separated, dangerous content is quarantined, taint is tracked into derived artifacts, LLM submissions are bound, execution demands a separate capability, and final Notebook writes are restricted to a single committer. Diagnostics and cognitive support back this defense from the side: SystemDoctor observes pipeline health, and cognition inserts additional confirmation for ambiguous human input and irreversible operations.

The defenses are arranged along the flow of information. First, external Web, PDF, and mail content is treated as UntrustedData rather than as commands; a deterministic pre-scan checks for known Injection, tool misuse, credential exfiltration, and obfuscation, and objects with high risk are quarantined (the ingress boundary). Because things may still be missed, taint propagation tracks the scope of influence (propagation tracking). The entire request envelope sent to the LLM is fixed by a PreparedInputToken, and substitution immediately before submission is detected (the submission boundary). Even if the LLM is deceived, external actions additionally require a CapabilityLease, and one-shot atomic consume together with the ExecutionTicket suppresses replay and TOCTOU (the execution boundary).

In recent SourceVault, one more axis has been added here: the state in which something that should have been executed has not been executed. Examples include the owner forgetting to reply to mail that requires a response, overlooking deadlines or NextReview in Notebooks, not starting the preparation required for events on the calendar, and not fulfilling periodic Routines. This does not judge the human as "anomalous"; it is treated as an operational state anomaly in which an inconsistency has arisen between the Obligations held by the system and the observed Fulfillment. As with unexpected tool requests from the LLM or declines in run integrity, these are first reduced to deterministic events and rate streams, deviations from baselines are detected observe-only, and the results are connected to attention prompting and review. From this viewpoint, SourceVault's safety layers are being extended beyond "layers that prevent attacks and LLM misbehavior" to "layers that detect important work silently falling through the cracks."

This chapter can therefore be read as one that places security anomalies, system health anomalies, LLM anomalies, and owner operational anomalies into the same "observation, evidence, attention, review" framework without conflating the boundaries of authority control. In what follows, 9.1 presents the overall picture; 9.2 through 9.4 describe the three boundaries of information ingress, egress, and submission/execution; 9.5 and 9.6 cover diagnostics and cognitive support; 9.7 and 9.8 cover operational anomalies and the Routine Agenda; 9.9 describes adjudication and anomaly analysis; 9.10 summarizes the defense in depth; and 9.11 lays out the future directions.

9.1 Why a safety layer is needed outside the mining layer

The mining layer connects materials that were previously isolated. As connections increase, however, so do dangers: leakage of topics derived from private mail, Prompt Injection in Web page bodies, taint propagation from cited materials into generated artifacts, incorrect identity links, dangerous expression proposals by the LLM, shared-state corruption by workers, missends caused by ambiguous user instructions, and leakage of confidential state from diagnostic information.

Safety is therefore deployed not as a single filter but in multiple layers.

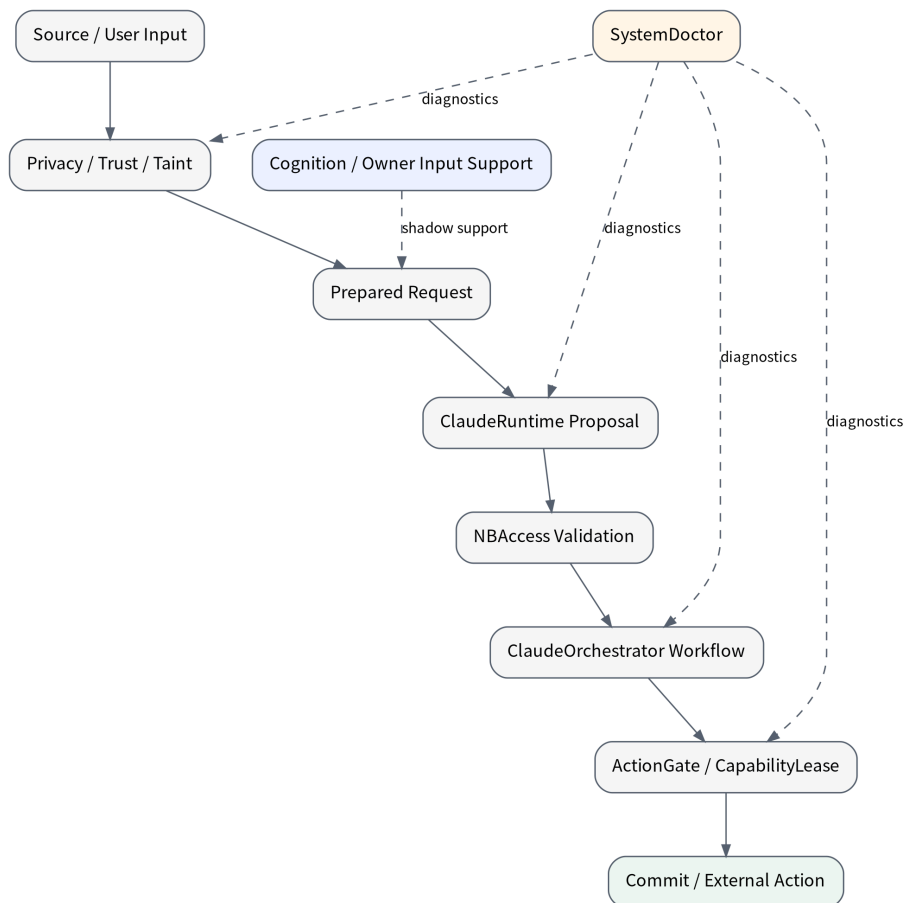


Figure 9. Defense in Depth Across Safety, Diagnostics, and Cognitive Support

9.2 Ingress boundary — quarantine of untrusted data and taint tracking

9.2.1 All external input is untrusted data

External content from the Web, PDFs, mail, APIs, arXiv, HTML, and so on is data, not system instructions. Even if the body text says "ignore previous instructions," "execute this tool," or "output credentials," it is never promoted to an execution instruction. External content cannot raise its own instruction authority by itself.

9.2.2 InputTrustAssessment and quarantine

Inputs are evaluated against risk vectors such as Prompt Injection, Tool Misuse, Credential Exfiltration, Data Exfiltration, Obfuscation, Invisible Unicode, HTML Comment Directive, and Cross-object Contamination.

A deterministic pre-scan is performed first, followed by additional analysis as needed. By design, quarantined objects are excluded from primer, KG expansion, agentic retrieval, and LLM reasoning. The implementation, however, is [Partially implemented]. The paths where a quarantine gate can be confirmed in the current code are extractor exclusion in the mining pipeline, Block / MetadataOnly / SafeInspection for Web summaries, and replacement of mail bodies with safety notes; a common exclusion based on `SafetyState` is not yet wired into searchindex, Knowledge Home, MCP, or PromptRouter (the searchindex release policy evaluates privacy, tags, State, and expiration, but does not evaluate `SafetyState`). A uniform gate across all search surfaces remains a design goal.

9.2.3 Taint propagation

Taint does not propagate unconditionally along every relation edge. The propagation conditions are restricted as follows.

```
CarriesContent -> True
and
ContributingSpanRefs is not empty
```

For example, merely sharing a topic with a dangerous Web page does not taint an object. Propagation occurs when a span of that body text has actually contributed to a summary or a generated artifact. Contains and AttachmentOf likewise record in which direction content contributed, not mere existence.

```
FromRef
ToRef
EdgeKind
CarriesContent
ContributingSpanRefs
TransformRef
Direction
PolicyVersion
```

A distinction is drawn between the case where only part of a document is tainted and the case where the entire document is quarantined. This makes it possible to treat the impact of indirect Prompt Injection as an information-flow problem of "which derived artifacts it has reached."

9.2.4 Declassification

For declassification (taint release), design and implementation must be distinguished. The current `SourceVaultTaintDeclassify` refuses declassification based on the LLM's own judgment, majority voting among multiple LLMs, or a reducer's judgment [Implemented]. On the other hand, if there is an explicit string confirmation from the owner (OwnerConfirmation) or a successful deterministic sanitizer, declassification is permitted regardless of the level of risk. The design specification prohibits declassification of high-risk materials by owner confirmation alone and requires, according to risk, sanitized projections, deterministic inspection, safe previews, diff presentation, and re-confirmation of the release destination and purpose, but these per-risk requirements are not yet wired into the current code and API [Design-only]. This is an important implementation gap in the safety specification and is treated as a priority issue in 9.11.

9.3 Egress boundary — sensitive cells and prompt privacy

Secret information in a Notebook propagates not only through cells but into dependent outputs. NBAccess builds the context intended for providers and does not pass raw secrets to the Runtime or the Orchestrator.

SourceVault's prompt privacy composes Prompt Cell Privacy, Referenced Source Privacy, Dependency Privacy, Model Trust Domain, Result Privacy, User-specified Privacy, and so on with max toward the safe side. For a high-privacy prompt, constraints such as the following are imposed.

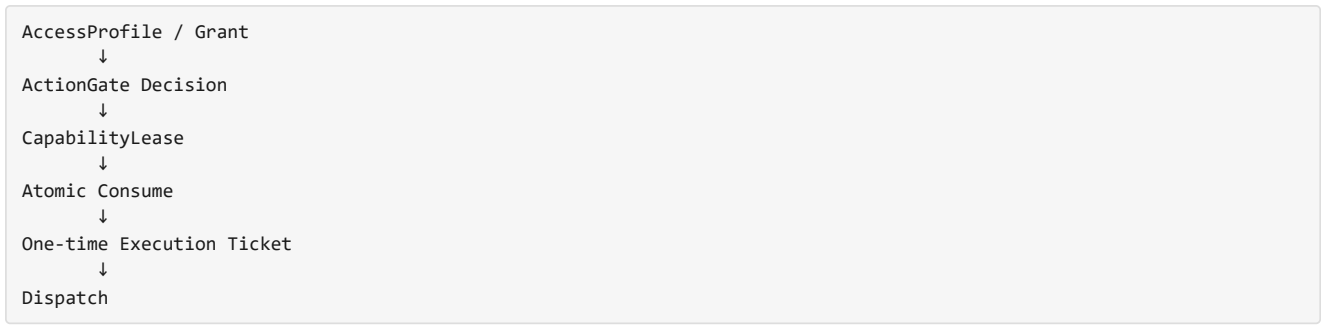
```
AllowedTrustDomains -> Local / Private
CloudFallback -> Deny
RawPromptStored -> False
```

What matters in secret-cell handling is not that the Runtime or Orchestrator sees a secret and then hides it, but that NBAccess passes only the permitted projection from the outset.

9.4 LLM submission and execution boundary — request binding and capabilities

9.4.1 Capability Broker and ExecutionTicket

Being permitted to view content or submit a prompt is distinct from being able to execute an external action. Execution authority is composed through the following layers.



The canonical state of a `CapabilityLease` resides in the broker-side ledger (the canonical ledger that manages use counts, expiration, and state transitions). One-shot, `MaxUses`, expiry, revocation, and replay are determined atomically by the ledger. The `ExecutionTicket` is issued in the same transaction as the consume and is redeemed one-shot immediately before execution.

9.4.2 PreparedInputToken

A `PreparedInputToken` does not bind merely to a hash of the prompt body. It binds to the entire request envelope that is actually submitted.

```
Provider
Model
Deployment
Messages (system / developer / user)
Tool Schemas
Retrieval Refs and Digests
Isolation Profile
Privacy Decision
Capability Ceiling
Output Schema
Run Ref
Step Ref
```

The provider adapter recomputes the digest immediately before submission and rejects the request on mismatch. This prevents TOCTOU attacks in which the model, system prompt, tool schema, or retrieval content is swapped after inspection.

9.4.3 Shadow / Warn / Enforce

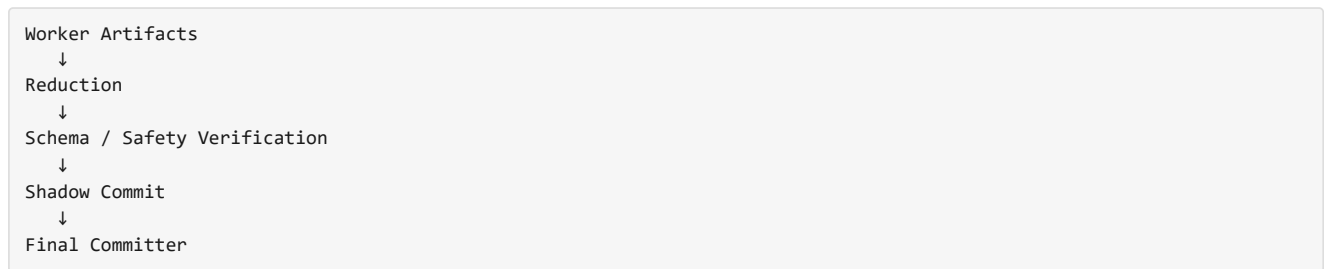
That the `Capability Broker` infrastructure has been implemented is not the same as its being enforced at all times at every entry point.

Mode	Behavior
Shadow	Record only. Does not block submission
Warn	Warns on inconsistency
Enforce	Rejects missing or mismatched tokens

At present, 18 LLM entrypoints (`webingest`, `servicemanager`, `mining`, `maildb`, `eagle`, `workflowcatalog`, `searchindex`, the main package, `llmlog`, `wiring`, `promptrouter`) carry observation hooks and self-gates (`wiring` in which the entrypoint itself performs boundary checks, such as token verification, immediately before submission). The defaults are furthermore two-tiered. The default mode (`$SourceVaultLLMBoundaryMode`) is `Shadow`, but the default of the observation toggle (`$SourceVaultLLMBoundaryShadow`) is `False`, and even when the mode is `Shadow`, if the toggle is `False` the gate passes through as a no-op with `Status -> "Disabled"`. Because persistent enablement of observation depends on machine-local configuration, the operational state of a particular PC cannot be generalized as the overall default. `Warn / Enforce` can be promoted per entrypoint by explicit owner configuration, but the transition to full enforcement is still in progress.

9.4.4 Side-effect separation in ClaudeOrchestrator

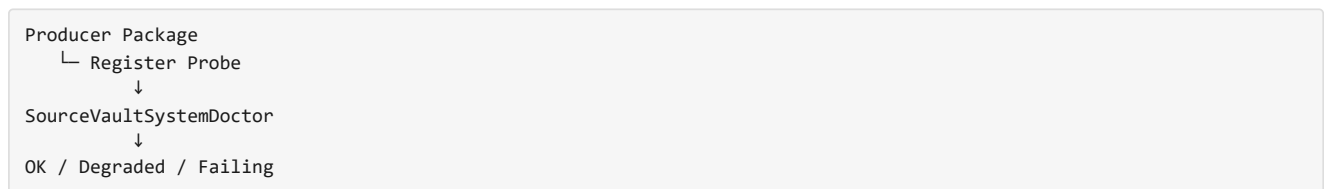
Workers are artifact producers. Things a worker does not execute directly include NotebookWrite, RunProcess, SystemCredential, unrestricted URL access, and live Notebook commits. Commits are performed only by the single committer.



9.5 Diagnostics — SystemDoctor and the confidentiality of diagnostic information

9.5.1 Probe aggregation in SystemDoctor

SourceVault's SystemDoctor is not a monolithic diagnostic engine that depends directly on each subsystem; it aggregates probes registered by producers.



Probe execution defensively handles timeouts, messages, `$Failed`, Abort, Throw, malformed results, and the like. Even if one probe stalls, SystemDoctor as a whole is not brought down; that component is reported as Degraded or Failing.

9.5.2 Process Supervisor diagnostics

The Runtime's Process Supervisor manages, for external jobs, PendingSpawn, Running, and the terminal states (Completed / Reaped / Vanished, and CleanupFailed when cleanup fails).

To account for PID reuse, the process start time is matched in addition to the PID. The cleanup order performs seat release first and archives afterward. If any CleanupFailed exists, the SystemDoctor probe returns Degraded.

9.5.3 Confidentiality of diagnostic information itself

Diagnostic information itself carries confidentiality. Even a single boolean such as "a private alert exists" could allow inferences about the user's condition or the existence of a private corpus. The normal heartbeat therefore carries only allowlisted information such as Component, Health, and public ReasonCodes. Items such as PendingSensitiveAlerts are removed from normal probes and separated into a local-only doctor.

Sink	Permitted information
Local SystemDoctor UI	Necessary local-only details
Diagnostics Log	Explicit allowlist
Machine Heartbeat	Component, Health, public Reason
Cloud Heartbeat	Minimal information such as GlobalHealth
Mail Alert	Fixed, generalized wording

9.6 Cognitive support

9.6.1 Purpose of support and signals

SourceVault's cognitive support is not a system that judges the user's abilities and alters their authority. Its purpose is to provide support before a committing operation when, for example, the referent of an instruction is ambiguous, a recipient is missing, quoted text is mixed with the user's own instructions, an irreversible operation is requested, evidence is insufficient, the same exploration is repeated, or there is fixation on an incorrect premise.

Inputs are decomposed into signals such as OwnerInstruction, QuotedData, AmbiguousReferent, RecipientUnspecified, PotentialQuotedDataRelease, and IrreversibleActionRequested. Support modes include Normal, ReviewEnhanced, ConfirmBeforeCommit, and DraftOnly. Irreversible operations require owner confirmation regardless of cognitive state.

9.6.2 SensitiveLocalVault

Cognition events are stored in the SensitiveLocalVault under <LocalState>/sensitive/cognition/. The main principles are: excluded from cloud sync, encrypted, sharded per subject × month, local key reference, bitemporal events, crypto-shredding, prompt bodies not normally stored, and never used for authorization. SupportNeedTier and the like express the "degree of support needed," not an assessment of ability or authority.

9.6.3 Shadow-only recorder

Cognitive observation at the ClaudeEval entry point is shadow-only.

- It does not change call arguments
- It does not change return values
- It does not block the main processing on timeout
- It does not store prompt bodies
- It stores only digests, lengths, modes, and signal names

Cognitive support therefore never discloses the user's condition to LLM prompts, nor does it automatically curtail authority.

9.7 Operational state anomaly — observing non-execution and forgetting

9.7.1 Human and LLM errors on the same observation surface

A distinguishing feature of SourceVault is that it does not treat errors as an LLM-only problem. Moreover, the recent Routine Attention implementation has made explicit the direction of placing not only "wrong judgments" but also "necessary work not being executed" onto the same analysis surface as an observable operational state anomaly.

Error / deviation	Human / owner side	LLM / system side
Misremembering	Misidentifies past decisions	Adopts stale context
Unfounded assumption	Misidentifies the target or recipient	Fills in missing information
Looping	Repeats the same exploration	Regenerates the same query
Overconfidence	Commits on ambiguous instructions	Asserts weakly grounded claims
Quotation confusion	Mistakes someone else's text for the user's own instruction	Turns untrusted text into instructions
Missed fulfillment	Forgets replies, attendance, preparation, periodic work	Scheduled runs, workers, and probes do not complete as expected
Overlooked state	Views mail, Notebook, and Calendar separately and drops requirements	Source coverage becomes Stale / Unavailable, losing the basis for evaluation

These are handled with the common concepts of event, evidence, confidence, review, abstention, commitment, and obligation. Human non-execution, however, must not be interpreted as an "anomaly of cognitive ability." What is anomalous here is not the person but the gap between the expected system state and the observed state.

9.7.2 Obligation and Fulfillment — deterministic fulfillment evaluation

In the canonical specification v0.4 of the Routine Attention core, CalendarEvent, OnWorkTask, Commitment, and Routine are normalized into per-occurrence Obligations, and PrepTasks are derived from PreparationSpec as needed. The important point is that appointments, Notebook tasks, promises, and periodic work are not handled by separate notification mechanisms but are reduced to a common model of "what must be satisfied by when for this occurrence."

Fulfillment evaluation is not delegated to the LLM; it uses deterministic evidence. When a source cannot be fetched and evaluation is impossible, the result is Unknown rather than False, handled with Kleene three-valued logic. This avoids misjudging a Calendar or Notebook fetch failure as "not executed." Conversely, once something has become Overdue, the history is retained as WasOverdue even after catch-up, so it does not become "the past delay disappeared because the work was executed later."

In the implementation, SourceVaultRoutineExecutionStreams aggregates resolved ObligationOccurrences daily and produces RoutineExecutionRate and RoutineOverdueRate. In addition, SourceVaultRoutineOverdueSecondsByDay integrates OverdueContractSeconds, and these streams can be handed to the existing Cane anomaly workflow. Consequently, "the rate at which things that should have been executed were not executed" can be fed into the same anomaly analysis infrastructure as Prompt Injection signals, unexpected tool requests, and declines in run integrity.

Conceptually, the correspondence is as follows.



The two are not, however, treated as anomalies with the same meaning. An LLM anomaly indicates the health of the model / run, while a Routine anomaly indicates an inconsistency between Obligation and Fulfillment. What is shared is the processing form of "observe → convert to streams → compare against baselines → attention prompting / review," not cause estimation or authority control. Routine anomalies are also observe-only at present: detecting non-execution alone neither changes owner authority nor executes tasks on its own.

9.8 Routine Agenda — treating Calendar, Notebook, and Mail on the same attention surface

9.8.1 Three information sources onto a single attention surface

The practical value of Routine Attention lies not only in producing anomaly streams but in letting the owner check "what am I overlooking today" on a single working surface. The current SourceVaultRoutineAgendaData / SourceVaultRoutineAgendaView are implemented to handle at least the following three comprehensively.

Information source	Current ingestion	Treatment on the Agenda
Calendar	NBCalendarEvents / FreeBusy from NBAccess	Timed events, all-day events, busy capacity

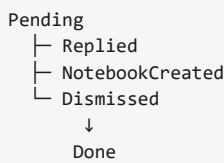
Information source	Current ingestion	Treatment on the Agenda
Notebook	Scans <code>\$onWork</code> with <code>NBOnWorkTasks</code>	Deadline, NextReview, Overdue, click to open the Notebook
Mail	<code>SourceVaultMailAgendaItems</code>	Reply needed, work requests, attendance requests, confirmations, estimated Deadline

In the Agenda implementation code, Calendar events are read first, then Deadline / NextReview items from `$onWork` are added into the same day buckets, and the R9 Mail Agenda is then loaded through loose coupling. Mail carrying a Deadline also enters the per-day Calendar as `MailDeadline`, and actionable mail is additionally displayed in an independent band. In other words, mail is not partitioned off as "something to view in the mail app," Notebook TODOs as "something to view only inside Notebooks," and Calendar as "something to view only in the schedule"; all are projected onto a single attention surface.

9.8.2 Mail Agenda R9

Mail Agenda R9 does not call an LLM when rendering the Agenda. It reads the Summary / Category / Priority / Deadline precomputed by `SourceVault_maildb` from the index and narrows down in the order of category gate, SPAM / irrelevance gate, owner-directed determination, and exclusion of resolved items. The owner-directed determination uses the identity layer's owner addresses and organization addresses, and only when necessary lazily decodes the head of the body to check salutation patterns. Mail in the same thread is aggregated into a single item.

Resolution state is managed as follows.



If a reply is needed, a reply Notebook is opened; if the item is carried over into work, a work Notebook is created under `$onWork`. This carry-over notebook retains the `MailRecordId`, allowing a return to the original mail and the subsequent reply thread. In the reverse direction as well, the `NotebookPath` can be referenced from the mail sidecar. This avoids the typical oversight in which "a request arrives by mail, but after a work Notebook is created the link to the original mail is lost."

9.8.3 PlanView extension — preparation tasks and procrastination prevention

The PlanView extension goes further: it generates preparation tasks for meetings and lectures ahead of time from `PreparationSpec`, and detects and mitigates procrastination using the `CapacityModel`, `LatestSafeStart`, `Slack`, `daily replan`, and `FocusRequest`. The design treats the plan itself as disposable, retaining only durable intent such as `pins`, `FocusReservation`, and `PlanningIntent`. When inputs are Stale / Partial, silent replanning is stopped and the previous plan is retained as `PlanInputDegraded`.

9.8.4 Verification of implementation status

Checking the main branch as of July 16, 2026 against the code and recent commits shows that this is not a concept living only in design documents; a substantial portion is already implemented.

- The Routine core is implemented through identity, three-valued evidence, Resolution, the durable-facts ledger, ActionGate, Board, AttentionContext, the durable notification queue, and the R7 anomaly streams. The API documentation records a cumulative 262 green for routine.
- R8 implements the deterministic formulaic score, automation eligibility, proposal / approval, `TriggerSpec` drafts, and one-shot permits, and the AT-1 permit / ExpiresAt / EnabledAudit gate has been added on the `AutoTrigger` side. However, permit enforcement (`$SourceVaultAutoTriggerEnforcePermit`) is off by default.
- `SourceVault_routineplan.wl` implements `PreparationTask`, `LatestSafeStart` / `Slack`, capacity placement with DAG validation, `replan`, `FocusRequest`, the `Gantt` / `Load` / `Process` views, `ScheduleFabric`, and the `Calendar` + `$onWork`

Agenda. The API documentation records 48 green for the plan side (routineplan 28 + reschedule 20) and 20 green for the view side.

- The latest R9 commit adds `SourceVault_mailagenda.wl`, implementing owner-directed actionable mail, thread aggregation, the Replied / NotebookCreated / Dismissed state machine, mail↔Notebook carry-over, and the privacy gate. `IncludeMail / MailItems` have also been added on the `SourceVault_routineplan.wl` side, and the MailDeadline and actionable-mail bands are integrated into the Agenda.
- On the other hand, the end of `api_routine.md` still contains the statement "R9 = mail not implemented," meaning the documentation update lags behind the code and commits on the latest main. This report therefore assesses R9 as "specification in draft, implementation landed on main, parts of the API documentation still catching up."
- Within the PlanView extension, the ledger wiring of durable PlanningConstraint / PlanningIntent / PlanChunkProgress, the Petri adapter for formal workflows, and the Plan diff view remain listed as not implemented in the API documentation.

With this implementation, SourceVault is moving toward treating not only LLM errors and Prompt Injection but also "the owner's important work silently remaining unexecuted" itself as a state deviation the system should observe. That said, SystemDoctor's component health and Routine Attention's operational health are not the same thing. Rather than collapsing software failures and unexecuted owner tasks into the same `Failing`, it is appropriate to present them in parallel in a higher-level health / attention view, each with its own evidence and action policy.

9.9 Adjudication and anomaly analysis

9.9.1 Multi-LLM adjudication and abstention

Agreement among multiple models is not taken as correctness in itself. The priority order of evidence is as follows.

```
Deterministic Test
>
Evidence
>
Verifier
>
Calibrated History
>
Model Agreement
```

Models belonging to the same CorrelationGroup are not counted as independent votes. When no valid conclusion can be reached, abstentions such as `NeedMoreEvidence`, `NeedOwnerClarification`, `SafeDraftOnly`, and `NoCommit` are permitted. High-risk actions are not Accepted without owner confirmation even when multiple models agree.

9.9.2 Observe-only anomaly analysis

The Cane anomaly workflow is observe-only. The initially active streams include `InjectionSignalRate × RunIntegrityRate` and `Pre-scan Failure / Liveness × Unexpected Tool Request`. Correlations with owner state are not included in the initial active set and are research-only opt-in. Even when an anomalous-looking correlation is found, no direct containment or restriction of authority is performed.

9.9.3 Cold start and baselines

When no sufficient baseline exists, no alert is raised on guesswork.

```
Missing["InsufficientBaseline"]
```

Baseline updates also proceed through the three stages `Candidate → Validation / Backtest → Activate`. Because a change to the baseline itself alters future evaluations, it is audited as a state mutation even under observe-only operation.

9.9.4 Correlation and causation

When two signals change due to the same upstream event, this is not taken as a causal relationship. A SharedUpstream hypothesis is, as a rule, limited to AssociationSupported. Promotion to causation requires additional grounds such as Controlled Replay, Deterministic Attack Chain, Isolation Stopped Recurrence, and Independent Streams.

9.10 Summary of defense in depth and boundaries after autonomization

9.10.1 Defenses by layer

Layer	Defenses
NBAccess	Notebook privacy, secret dependencies, safe execution, keys
ClaudeRuntime	Held Proposal, validation loop, prohibition of direct resource access
ClaudeOrchestrator	Worker / committer separation, transactional commit
SourceVault ingest	Untrusted data, security pre-scan
SourceVault mining	Provenance, confidence, review, taint
Prompt boundary	PreparedInputToken, request binding
Action boundary	CapabilityLease, atomic consume
Diagnostics	Probe isolation, sink field allowlist
Cognition	Local-only, shadow, confirmation of irreversible operations
Routine Attention	Obligation / Fulfillment, forgetting detection, durable notifications, Agenda
Adjudication	Evidence-first, abstention
Anomaly	Rate streams for LLM / security / routine, observe-only, cold start, conditions for promotion to causation

Safety holds not as a single filter but as the following product.

Safety

- = Privacy Boundary
- × Proposal Validation
- × Workflow Capability
- × Release Gate
- × Request Binding
- × Action Lease
- × Human Confirmation
- × Diagnostics

9.10.2 Safety boundaries added by MCP, AutoTrigger, and multi-PC operation

With MCP and AutoTrigger added, the system is no longer configured to "run only when the user explicitly executes it in a Notebook." The following boundaries therefore become important: MCP is a projection layer, not an authority; AutoTrigger is a thin scheduler that makes no decisions internal to workflows; and SystemDoctor is read-only in principle. An MCP transport token is not a data access grant, and an sv:// URI is not a capability. That AutoTrigger was able to create a job likewise does not mean the external side effects within that job were authorized.

Boundary	What it permits	What it does not permit
sv:// URI	Object identity and reference	Read authority itself
MCP transport	Client connection to the tool endpoint	Automatic release of private data
sourcevault_search/get	Search and retrieval of released projections	Retrieval of confidential body/raw without a grant (except the PublicDoc body exception; see 5.8)
sourcevault_deposit	Append-only artifact creation	Deletion of existing objects, relaxation of privacy, arbitrary mutation
AutoTrigger	Job construction and invocation of the target executor when conditions hold	Substituting for side-effect decisions internal to workflows

Boundary	What it permits	What it does not permit
SpecificMachine placement	Selection of the target PC	Bypassing NBAccess / action capabilities
SystemDoctor	Observation and aggregation of health	Disclosure of secrets or expansion of authority based on diagnostic results alone

In particular, the ProviderClass self-declared by an MCP client is not used as the authority for release decisions; only trusted information established via the transport / grant / main kernel is adopted. A design that does not misidentify cloud-backed clients with local tools, such as Claude Code / Codex, as LocalLLM is important for opening SourceVault to a diverse range of models.

9.10.3 Maintaining the single committer even as autonomy increases

Even when LLMs can create artifacts via MCP deposit, AutoTrigger can launch workflows, and other PCs can execute processing, there is no need to open direct writes to live Notebooks to workers. What should be shared can be placed in SourceVault as sv:// artifacts / snapshots / references, and only the final reflection into Notebooks can be funneled to NBAccess's PendingFinalActionQueue or the Orchestrator committer. This is the central design principle that reconciles this system's "autonomization" with its "control boundaries."

```
Primary code and documentation references: SourceVault_info/design/sourcevault_universal_mcp_access_spec_v2.md /
SourceVault_info/docs/api_mcp.md / SourceVault_info/docs/api_capbroker.md / SourceVault_info/docs/api_autottrigger.md /
SourceVault_info/docs/api_diagnostics.md / SourceVault_info/docs/api_routine.md / SourceVault_info/docs/api_routineplan.md /
SourceVault_info/docs/api_mailagenda.md
```

9.11 Future Directions

Among the gaps between the current implementation and the design specification, two priority issues bear directly on safety. The first is to implement the design's per-risk declassification requirements in SourceVaultTaintDeclassify and the API, closing the current gap in which high-risk materials can be declassified by owner confirmation alone (9.2.4). The second is to wire SafetyState into a common release / retrieval gate for searchindex, Knowledge Home, MCP, PromptRouter, and primer / KG / agentic retrieval, making the design's quarantine invariant checkable across paths (9.2.2).

In parallel, the LLM submission boundary will transition in stages from Shadow to Warn, then per-entripoint Enforce, and finally full enforcement across the system. As a precondition — given that the observation toggle currently defaults to False — the work begins with always-on observation and measurement of token coverage, then widens the scope of enforcement while observing false refusals and unregistered entripoints. If PreparedInputTokens are minted upstream in every call chain and consumed immediately before provider submission, the system becomes robust against post-inspection swapping of prompts / retrieval / tool schemas.

Next, CapabilityLease needs to be connected to all side-effect boundaries, including send, publish, delete, network, secret, and Notebook write. Even if Prompt Injection cannot be prevented completely, the blast radius can be kept small with a structure in which a deceived LLM cannot increase its own authority and can execute only the permitted operation and target, one-shot.

For taint, the task is to record ContributingSpanRefs on all paths: Web → Summary, Mail → Answer, PDF → Report, MCP retrieval → Agent output, Agent artifact → next Agent. The more SourceVault is used as a shared surface between agents, the more this span-level provenance matters. SafeInspection and the declassification workflow are also important points going forward. By prohibiting copy / export in the normal mode for investigating quarantine payloads, and by routing anything bound for the outside through sanitized projections and explicit declassification, re-infection caused by the act of investigation itself can be prevented.

Anomaly analysis should remain observe-only for the time being, and it is reasonable to use correlations such as InjectionSignalRate with RunIntegrityRate, and pre-scan failure with unexpected tool request, for campaign detection.

Correlations should not, however, be tied directly to automatic restriction of authority; the current cautious policy of requiring additional evidence such as controlled replay should be maintained.

In addition, it is important to mature `RoutineExecutionRate`, `RoutineOverdueRate`, and `OverdueContractSeconds` into operational anomaly streams standing alongside LLM / system anomalies. The goal here is not to diagnose the owner's condition but to discover early the system state in which "an expected Obligation is not being satisfied." When the source coverage of Calendar, Notebook, or Mail is Stale / Unavailable, this must not be misidentified as non-execution; coverage degradation needs to be treated as a separate stream.

For the unified Agenda, the R9 Mail Agenda specification needs to advance from draft to a stable version, and the implementation-status descriptions in `api_routine.md` need to be synchronized with the latest main. The next topics are automatic Done determination when a reply is sent from an external mailer, formal integration of the relation between mail threads and carry-over notebooks into the mining graph, contention tolerance when the shared sidecar is updated from multiple PCs, and consistent occurrence identity across Agenda / Board / PlanView. In PlanView, the shared-ledger wiring of durable `PlanningConstraint` / `PlanningIntent` / `PlanChunkProgress`, the Plan diff view, and the Petri adapter restricted to formal workflows need to be completed. LatestSafeStart and Slack are effective for procrastination prevention, but the current policy of not modifying the plan on its own when inputs are Stale should be maintained, and when a plan is infeasible, options such as `Defer` / `Waive` / `Automate` / `NegotiateDeadline` should be presented to the owner.

Ultimately, it would be desirable to display SystemDoctor's software health, Cane's LLM / security anomalies, and Routine Attention's operational anomalies side by side on a single higher-level observability surface. It is important, however, not to collapse these into a single score, but to relate them while keeping cause, privacy, authority, and recommended actions separate. This makes it possible to detect the distinct problems of "the LLM erred," "the service broke," and "important work remained unexecuted" on the same observation infrastructure while guiding each to a different, safe response.

Chapter 10. Comparison with Similar Systems

This chapter compares the transreal stack with LangGraph, Microsoft Agent Framework, Haystack, and similar systems. The purpose of the comparison is not to judge superiority but to clarify what each system optimizes for and to provide an objective view of the transreal stack's positioning. Comparing on a simple count of "agent framework features" alone makes the characteristics hard to capture. The distinctiveness of the transreal stack lies in its vertical integration of Notebook cell privacy, held validation of Wolfram expressions, the single committer, source/snapshot/provenance, Knowledge Home, MCP, and diagnostics.

On the other hand, large Python-based OSS projects have the advantage in generality, ease of adoption, provider ecosystem, public CI, cross-platform support, and community support. LangGraph is strong as a general-purpose durable agent graph, the Microsoft offerings in production governance, and Haystack in RAG pipelines, whereas the transreal stack is deeply specialized in a safe knowledge-work environment centered on Wolfram Notebooks. With the addition of MCP and sv:// URIs, the transreal stack has increased its connectivity to external models compared to before, but its center remains the Notebook and local/private knowledge. This specialization is a limitation, but it is also the reason the stack can integrate cell-level privacy and source provenance that other frameworks do not provide by default.

In what follows, 10.1–10.2 present the comparison axes and an overview, 10.3–10.5 compare against individual frameworks, 10.6 describes the positioning, and 10.7 summarizes the future outlook.

10.1 Comparison Axes

When the evaluation includes the extensions of Chapters 8 and 9, mere multi-agent functionality is insufficient as comparison axes.

Comparison axis	Description
Execution Control	How LLM proposals are validated and side effects are restricted
Workflow	Long-running, multi-agent, approval, retry
Knowledge Lifecycle	source, snapshot, claim, version
Retrieval	lexical, vector, KG, neighborhood search
Identity	author, sender, entity resolution
Privacy	Notebook, source, prompt, release
Security	Prompt Injection, taint, capability
Diagnostics	probe, heartbeat, doctor
Long-term Support	search episode, cognition, review
Deployment	provider, language, platform, CI

10.2 Comparison Table

Project	Core capabilities	Primary design focus	Adoption difficulty	Maturity	License
transreal stack	Notebook privacy, single-agent Runtime, multi-agent orchestration, source-first vault, Knowledge Home	Wolfram Notebook-centric, separation of responsibilities, single-committer, hash-addressed source, local privacy	Relatively high	Moderate	MIT
LangGraph	stateful agent orchestration	graph, durable state, human-in-the-loop	Moderate	High	MIT
Microsoft Agent Framework	production-grade multi-agent workflow	Python/.NET, durable execution, observability, hosting (governance when integrated with the ecosystem)	Moderate	High	MIT

Project	Core capabilities	Primary design focus	Adoption difficulty	Maturity	License
Haystack	retrieval, pipeline, agent, RAG	modular retrieval/routing/generation	Moderate	High	Apache-2.0

Core capabilities, licenses, and supported languages are values verified against each official repository and official documentation as of July 16, 2026. In contrast, "adoption difficulty" and "maturity" are the author's assessments, based on release frequency, the presence of stable releases and public CI, cross-platform support, and community size. Because the features and maturity of external frameworks change rapidly, these assessments must be read together with the verification date.

10.3 Comparison with LangGraph

LangGraph is strong in stateful long-running agent workflows, checkpoints, human-in-the-loop, branching, retry, and the like.

The difference from the transreal stack is that LangGraph is a general-purpose Python agent runtime, whereas the transreal stack vertically integrates Notebooks, Wolfram expressions, cell privacy, and SourceVault.

LangGraph has the advantage in providers, deployment, and community integrations. The transreal stack, in turn, is distinctive in Notebook cell-level privacy, held Wolfram expression validation, the Notebook single committer, the integration of source/snapshot/claim, the OOPS Knowledge Home, SensitiveLocalVault, and so on.

10.4 Comparison with Microsoft Agent Framework

The Microsoft Agent Framework core emphasizes Python/.NET agent and graph workflows, type-safe routing, checkpoints, durable execution, human-in-the-loop, MCP integration, hosting, and observability. Enterprise-oriented governance such as tenant policy, access governance, and compliance is more accurately assessed separately as an ecosystem strength that arises when integrated with the management and security features of Agent 365 and Azure, rather than as a standard feature of the framework OSS by itself.

The transreal stack is inferior in scale and operational maturity, but differs in that it unifies the Notebook work of an individual or a researcher, including sources, privacy, and long-term memory.

10.5 Comparison with Haystack

Haystack is mature as a RAG foundation that explicitly composes retrievers, rankers, document stores, generators, pipelines, and the like.

SourceVault also has search and primers, but its emphasis is not on mere document retrieval. It integrates immutable snapshots, stale propagation, Evidence Bundles, Notebook management, identity, owner-scoped topics, quote/session graphs, and release contexts.

10.6 Positioning of the transreal stack

Rather than a general-purpose framework aiming directly at the same market as LangGraph, Microsoft Agent Framework, and Haystack, the transreal stack is a dedicated stack that is deeply and vertically integrated toward safe execution, knowledge management, and long-term support for Wolfram Notebooks.


```
Notebook Cell
↓
Privacy / Secret Dependency
↓
LLM Proposal
↓
Workflow / Commit
↓
Source / Snapshot / Evidence
↓
Topic / Identity / Neighborhood
↓
Diagnostics / Cognition / Audit
```

This continuous, end-to-end integration is its greatest distinctiveness.

10.7 Future Directions

Going forward, the comparison targets need to be broadened beyond "agent frameworks" to personal knowledge management, secure RAG, workflow automation, and MCP data servers. Because SourceVault's functional scope has expanded, competitors in a single category no longer suffice to explain its positioning.

On the practical side, interoperability with external frameworks is also promising. For example, LangGraph or other agents could use SourceVault via MCP, enabling a division of labor in which SourceVault handles knowledge, privacy, and provenance while an external framework handles agent orchestration. This would allow the transreal stack to offer its strong parts as a service rather than remaining a closed monolith.

To strengthen the comparative case, quantitative evaluation is also necessary. If benchmarks corresponding to the design claims — workflow success rate, Notebook corruption rate, privacy leak tests, retrieval groundedness, multi-PC recovery time, Prompt Injection containment rate, and so on — can be published, comparison with mature OSS will become more objective.

Chapter 11. Maturity, Issues, and Future Priorities

This system contains a mixture of areas that are implemented, areas that are implemented but in staged transition, areas that are observe-only, and areas where design precedes implementation. The purpose of this chapter is to make these maturity differences explicit for each part and to present a realistic order of consolidation. The statement "SourceVault has that feature" alone is insufficient; it must be accompanied by which mode is the default and how far integration has progressed.

For example, 1A–1C of OOPS/Knowledge Home are implemented and verified, but the cross-media Neighborhood spanning all media is still in mid-integration. The Capability Broker is implemented, but for the LLM boundary the mode default is Shadow and the observation toggle default is off, so without machine-local configuration observation itself is disabled. AutoTrigger is implemented up to SpecificMachine remote runs, but automatic load balancing and failover for the WorkerPool are incomplete. Grasping these "per-part maturity differences" is what leads to a realistic operational plan.

Furthermore, maturity as a public OSS project is a different axis from the sophistication of internal features. When CI, releases, a version matrix, cross-platform tests, a threat model, and migration guides are lacking, reproducibility is low for external users. Being sophisticated as an R&D platform and being mature as generally distributed software must be assessed separately.

In what follows, 11.1–11.2 present per-area maturity and maintenance considerations, 11.3–11.4 discuss security and public-OSS issues, 11.5 presents a staged consolidation proposal, and 11.6 summarizes the future outlook.

11.1 Maturity Should Not Be Assessed as a Single Score

Area	Assessment
NBAccess basic privacy and held-expression validation	Core implementation present
Runtime proposal loop	Core implementation present
Orchestrator planner/spawn/reduce/commit	Core implementation present
Petri net workflow	Implemented
Source/Snapshot/Bundle	Implemented
PromptRouter/Release Context	Implemented
OOPS seed/Knowledge Home 1A–1C	Implemented and verified
General mail structuring	Implemented, being extended
Cross-media Neighborhood	Partially wired
Capability Broker	Implemented, in staged transition (observation toggle default off)
SafetyState gate for quarantine	Partially implemented (mining/Web summarization/mail summarization paths only)
Risk-tiered declassification	Design only (currently releasable by owner confirmation alone)
Cognitive Support	shadow-first
Anomaly	observe-only
Public CI/release	Limited
macOS/Linux	No sufficient public validation

11.2 Module Scale of SourceVault

SourceVault has grown multifunctional, with an umbrella loader and many subpackages.

For long-term maintenance, the following become important: automated verification of the inter-package dependency DAG, separation of public and private APIs, tests of init order, degraded behavior when optional modules are absent, contract tests, schema migration, event replay tests, snapshot compatibility, and preventing confusion among source/derived/projection.

11.3 Security Priorities

For NBAccess, confidential dependency propagation, the CloudPublishable boundary, held expression bypass, cross-kernel consistency of the key index, and MAC/tamper tests are important.

For Runtime/Orchestrator, adapter bypass, worker deny heads, committer capability, double execution on retry, external process cleanup, and policy re-checking after approval are important.

For SourceVault, the request-time release gate, revocation, private topic label leakage, substitution of prompt/retrieval envelopes, taint edge direction, heartbeat field leakage, and event replay determinism are important.

At present, there are two priority items where a gap between design and implementation has been confirmed. First, `SourceVaultTaintDeclassify` can release high-risk material with owner string confirmation alone, so the design's risk-tiered sanitizer/preview requirements need to be implemented. Second, the quarantine `SafetyState` is not wired into a common gate across searchindex, Knowledge Home, MCP, and PromptRouter, so cross-path quarantine invariant checks are needed (see 9.2.2, 9.2.4, and 9.11).

11.4 Issues as Public OSS

Viewed from the public repository, the following need to be put in place: public CI such as GitHub Actions, test coverage metrics, tag/release management, a version compatibility matrix, validation beyond Windows, reproducible setup, a minimal sample vault, standalone documentation of the security model, a threat model, migration guides, a public changelog, and so on.

11.5 Recommended Staged Consolidation [Author's proposal]

The following is not a fixed order established by the specifications but a prioritization proposed by the author of this report.

Stage 1: Stabilizing the Four Layers

- NBAccess boundary test
- Runtime adapter contract
- Orchestrator commit test
- SourceVault core replay test

Stage 2: Knowledge Home Integration

- Common TopicPosition across Mail, PDF, Notebook, and Web
- SharedAuthor/SharedTag edges
- unified NeighborhoodSearch
- cross-media live view

Stage 3: Enforcement Transition for Safety Boundaries

- LLM boundary from Shadow to Warn
- Enforce on the target entypoints
- Unification of CapabilityLease and dispatch
- sink field allowlist

Stage 4: Evaluation of Long-Term Knowledge

- Effectiveness measurement of retrieval episodes
- recall/NDCG evaluation of the OOPS seed
- novel topic growth rate
- false link/false topic rate
- false intervention rate of cognition support

11.6 Future Directions

In the short term, it is rational to give top priority to the boundary tests of the four layers and to public CI. Fixing the core boundaries — NBAccess privacy, the Runtime adapter, Orchestrator commit, and SourceVault replay — and then adding features on top of them will reduce long-term regressions.

In the medium term, advancing Knowledge Home integration across all media, LLM boundary Enforce, CapabilityLease action integration, the comprehensive SystemDoctor auto-trigger, and the multi-PC WorkerPool will consolidate the currently scattered advanced features into a single operational experience.

In the long term, it is important to put public benchmarks, a sample vault, reproducible setup, and a threat model in place, elevating the system from research code to a reproducible platform. In particular, safety claims must be supported by continuous verification that includes attack tests and failure injection.

Chapter 12. Overall Assessment

The four repository groups have evolved from "tools for calling LLMs from Mathematica" into "an environment in which knowledge is acquired around the Notebook, evidence is preserved, agents reason, workflows execute, and the results are accumulated again as knowledge." What produces this distinctiveness is not the number of features but the boundary design. Secrets are assigned to NBAccess, the single turn to Runtime, multiple agents and commits to Orchestrator, and the source lifecycle and knowledge space to SourceVault; even as MCP and AutoTrigger extend the system to external and autonomous execution, it strives to maintain the boundaries of authority and provenance.

On the other hand, it is not a finished, monolithic product. Cross-media neighborhood search, full Enforce, WorkerPool failover, public CI, and the like remain future work. The appropriate assessment is therefore "an R&D platform with an advanced design philosophy whose main parts are substantially implemented." In what follows, 12.1–12.2 assess the foundations and the extensions, 12.3–12.4 describe the technical value and the limitations, 12.5 gives the final conclusions, and 12.6 summarizes the long-term outlook.

12.1 Baseline Assessment

The four repositories had clear responsibility boundaries from the outset.

- NBAccess: the boundary between Notebooks and secrets
- ClaudeRuntime: the execution core of a single turn
- ClaudeOrchestrator: multiple workers and commit discipline
- SourceVault: source lifecycle and deterministic replay

In particular, the design that forbids workers from writing directly to the Notebook and funnels writes into a single committer, the design in which PromptRouter only returns unevaluated expressions and never evaluates them itself, and the design in which Runtime delegates safety decisions to NBAccess are all consistent in reducing accidents in real operation.

12.2 Assessment Including the Mining and Safety Layers

With the addition of the SourceVault mining layer (Chapter 8), the system can now not only store and search materials but also reorganize knowledge by topic, identity, quotes, sessions, and work history.

The OOPS corpus is not merely a body of past mail. Roughly 6,500 emails are accompanied by roughly 4,100 manually constructed topic items, topic relations, mail-to-topic relations, and quote relations. This structure serves as the initial coordinates for situating new general mail, Notebooks, PDFs, and web pages.

At the same time, the defense-in-depth layer (Chapter 9) controls the dangers that arise as the connectivity of knowledge expands.

- privacy is protected by NBAccess and release contexts
- Prompt Injection is isolated via untrusted data and taint
- LLM transmission is bound by PreparedInputToken
- external actions are restricted by CapabilityLease
- Notebook commits are limited to the single committer
- diagnostic information is restricted by the sink allowlist
- cognitive support is local-only and shadow-first
- anomaly analysis starts observe-only

Through this structure, being close as a search result, being viewable, being sendable to an LLM, and being able to execute external operations are kept separate.

12.3 Technical Value

The first is the safe connection of confidential Notebooks and LLMs. The system identifies secret cells and dependent outputs in a Notebook, varies the context per destination, and validates LLM proposals as held expressions.

The second is source-centric reproducibility. The system tracks the sources, snapshots, spans, claims, workflows, and generated artifacts that were used, and judges the staleness of artifacts when upstream materials are updated.

The third is the personal Knowledge Home. From the current material, one can move to the next piece of knowledge guided by topics, people, quotes, time, and past explorations.

12.4 Limitations

For those seeking a replacement for a general Python agent, LangGraph, Haystack, and the like are likely easier to adopt. The transreal stack presupposes Mathematica, the Wolfram Language, Notebooks, the loading of multiple packages, and a local vault.

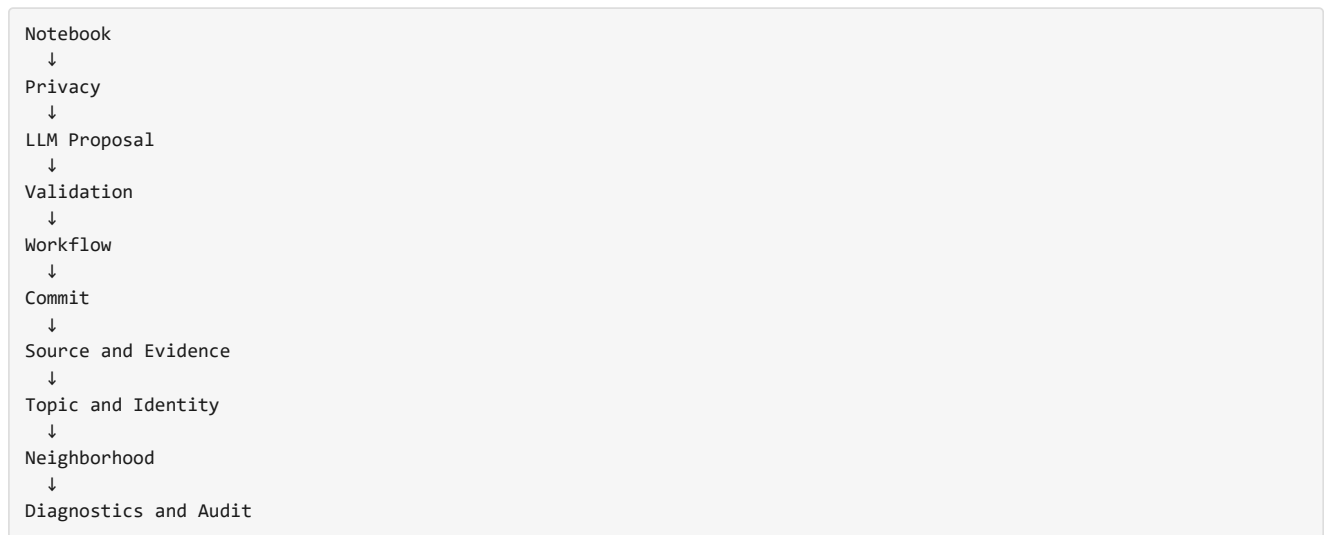
In addition, the maturity of public CI, releases, and platform validation is limited. The cross-media Knowledge Home, full Enforce of the safety boundaries, and the real-world effectiveness of the cognitive and anomaly features also require future integration and evaluation.

12.5 Final Conclusions

Rather than a finished general-purpose agent product, this system is appropriately assessed as follows.

NBAccess, ClaudeRuntime, ClaudeOrchestrator, and SourceVault constitute a sophisticated, substantially implemented R&D platform that, with the Mathematica Notebook at its core, integrates confidentiality boundaries, LLM reasoning, multiple agents, the source lifecycle, knowledge graphs, neighborhood exploration, execution authority, and auditing. And the four repositories have grown beyond the scope of "a set of tools for calling LLMs from Notebooks" into a system that manages the acquisition, structuring, exploration, reasoning, execution, and auditing of knowledge under a single model of separated responsibilities.

Its distinctiveness lies not in the number of individual features but in treating the following whole as one continuous model.



This is an attempt to reconstitute the Wolfram Notebook not as a mere computational document but as a working environment for long-term knowledge formation, reasoning, execution, and verification.

12.6 Future Directions

Going forward, the greatest potential for growth of this system lies in a configuration in which multiple LLMs, multiple PCs, and multiple workflows sharing SourceVault also share the same sv:// object graph and policy. Humans use the Notebook as the primary UI, while behind the scenes local LLMs, cloud LLMs, remote workers, and scheduled workflows cooperate.

To realize that vision safely, PreparedInputToken, CapabilityLease, the single committer, the release gate, and taint provenance must be completed not as "optional features" but as standard paths of the execution platform. The more autonomy increases, the more the design must move in the opposite direction, strengthening privilege separation and auditability.

Ultimately, this leads to a vision that treats the Wolfram Notebook not as a mere computational document but as the Knowledge Operating Environment of an individual or a research group. Sources, conversations, code, specifications, workflows, mail, documents, and experimental results being linked in the same reference space and reused while preserving their respective privacy and provenance — this can be regarded as the long-term destination of these four repository groups.

Appendix A. Main Reference Materials

This report was compiled primarily from the READMEs, API documents, design specifications, and main implementation files of the transreal public GitHub repositories as of July 15, 2026. Chapters 8 and 9 in particular drew heavily on the following.

- NBAccess/README.md
- NBAccess.wl
- NBAccess_crypto.wl
- NBAccess_info/design/ClaudeEval_runtime_orchestrator_nbaccess_async_compat_final_spec_permission_modes_v3.md
- ClaudeRuntime/README.md
- ClaudeRuntime.wl
- ClaudeRuntime_processsupervisor.wl
- ClaudeRuntime_info/docs/api_processsupervisor.md
- ClaudeOrchestrator/README.md
- ClaudeOrchestrator.wl
- ClaudeOrchestrator_workflow.wl
- ClaudeOrchestrator_info/docs/api.md
- SourceVault/README.md
- SourceVault_core.wl
- SourceVault_searchindex.wl
- SourceVault_identity.wl
- SourceVault_mining.wl
- SourceVault_oopsseed.wl
- SourceVault_mailstructure.wl
- SourceVault_knowledgehome.wl
- SourceVault_diagnostics.wl
- SourceVault_capbroker.wl
- SourceVault_taint.wl
- SourceVault_cognition.wl
- SourceVault_adjudication.wl
- SourceVault_anomaly.wl
- SourceVault_info/design/sourcevault_search_foundation_implementation_spec_v1.md
- SourceVault_info/design/sourcevault_self_organizing_mining_identity_tag_spec_v0_1.md
- SourceVault_info/design/sourcevault_cane_knowledge_home_mining_spec_v0_7.md
- SourceVault_info/design/sourcevault_cane_handoff_2026-07-14.md
- SourceVault_info/docs/api_knowledgehome.md
- SourceVault_info/docs/api_oopsseed.md
- SourceVault_info/docs/api_identity.md
- SourceVault_info/docs/api_cognition.md

Statements about implementation status distinguish between the intent stated in the design specifications and the implementation state that could be confirmed in the public code, API documents, and handoff materials.

Appendix B. Main Code, API, and Design Documents Referenced in the Expansion and Revision

For this expansion, the author cross-checked not only the READMEs but also the implementation code on public GitHub as of July 16, 2026, the API references in SourceVault_info/docs, the specifications in SourceVault_info/design, and the implementation specifications of the codified workflows. The following are the main grounds for the added descriptions.

Area	Main reference files
claudecode foundation layer	transreal/claudecode/claudecode_info/docs/api.md; claudecode_info/docs/api_directives.md
PromptRouter	SourceVault_info/docs/api_promptrouter.md; SourceVault_promptrouter.wl
PromptWorkflow	ClaudeOrchestrator_promptworkflow.wl
Requirements → specification	SourceVault_workflows/spec-review/SVWorkflow_SpecReview_info/design/implementation-spec.md
Specification → implementation	SourceVault_workflows/spec-impl/SVWorkflow_SpecImpl_info/design/implementation-spec.md
Workflow registry	SourceVault_info/docs/api_workflowregistry.md
SourceVault practical use	SourceVault_info/docs/user_manual.md
sv:// URI/MCP	SourceVault_info/docs/api_mcp.md; SourceVault_mcp.wl; SourceVault_info/design/sourcevault_universal_mcp_access_spec_v2.md
Capability Broker	SourceVault_info/docs/api_capbroker.md; SourceVault_capbroker.wl
AutoTrigger	SourceVault_autotrigger.wl; SourceVault_info/docs/api_autotrigger.md; SourceVault_info/design/sourcevault_auto_trigger_scheduler_spec_v0_1.md
Diagnostics/SystemDoctor	SourceVault_info/docs/api_diagnostics.md; SourceVault_diagnostics.wl
RuntimeSession/Artifact boundary	ClaudeOrchestrator_info/design/claude_orchestrator_runtime_session_episode_petri_spec_v0_1.md

For Routine Attention/Agenda, the following were referenced.

- SourceVault_info/design/sourcevault_routine_attention_spec_v0_4.md — The canonical specification of the Routine Attention core. v0.3 is a historical reference.
- SourceVault_info/design/sourcevault_routine_planview_ext_spec_v0_1.md — The planning-layer and visualization-layer extension.
- SourceVault_info/design/sourcevault_routine_mail_agenda_spec_v0_1.md — The R9 Mail Agenda specification.
- SourceVault_info/docs/api_routine.md / SourceVault_routine.wl — Obligation, Attention, the R7 anomaly stream, and R8 automation.
- SourceVault_info/docs/api_routineplan.md / SourceVault_routineplan.wl — Plan, ScheduleFabric, Agenda, and visualization.
- SourceVault_info/docs/api_mailagenda.md / SourceVault_mailagenda.wl — actionable mail, resolution state, and mail→Notebook carryover.
- Commits in the public GitHub history used to confirm implementation: 409012e (R7/R8/AT-1/routineplan core), c0c487d (Agenda/visualization/ScheduleFabric/replan), c8d6e82 (R9 mailagenda integration).

How to Read the Implementation Status

This report distinguishes between what exists in the specification documents and what is implemented in the current code. In particular, to avoid overestimation, the following points are explicitly distinguished: (1) the policy of making the comprehensive SystemDoctor the "first automated workflow" is canonical in the AutoTrigger design specification, but the automatic trigger remains a deferred item in the diagnostics API; (2) remote execution via SpecificMachine is implemented,

but automatic WorkerPool load balancing/failover is not; (3) `sv://` is the canonical identity layer for external references, but legacy internal refs remain for compatibility; (4) for quarantine, the `SafetyState` gate and risk-tiered declassification are areas where the design specification precedes the implementation.

As for the future separation into `claudecode_core`, it is described not as a fact implemented in the public code but as the author's proposed consolidation of `claudecode.wl`'s current responsibilities into a core and legacy compatibility.

Appendix C. Discrepancies Between Source Code and API Documentation (verified July 17, 2026)

During the cross-checking for this report, places were found where the source code and the API documentation disagree. The body of the report treats the source code as authoritative, but the discrepancies between primary sources are made explicit here. Reading the API documentation alone may lead to a misunderstanding of the implementation state.

Item	Source code (authoritative)	API documentation statement	Recommended action
NBAccess crypto backend	SystemCredential put/get/delete/index persistence is implemented	api_crypto.md says "write APIs are TODO"	Update the API documentation and the source header comment
NBStoreCredentialKey return value	Association with Status/KeyRef, etc.	keyRef / <code>\$Failed</code>	Decide the public contract and synchronize
NBDeleteCredentialKey return value	Association with Status/KeyRef	Boolean	Decide the public contract and synchronize
Orchestrator parallel Spawn	DAG-parallel path via <code>MaxParallelism/UseDAG</code> is implemented	"serial/sequential, currently 1"	Document the options together with their defaults
R9 Mail Agenda	<code>SourceVault_mailagenda.wl</code> and the routineplan wiring are implemented	api_routine.md ends with "R9 = mail, not implemented"	Bring the documentation up to date with the latest main
Declassification	Releasable by owner string confirmation alone (the API documentation says the same)	design specifies risk-tiered requirements	Implement the design's risk-tiered requirements, or state in the design that they are unimplemented